A Multi-Domain Survey on Time-Criticality in Cloud Computing

R. Andreoli, R. Mini, P. Skarin, H. Gustafsson, J. Harmatos, L. Abeni, and T. Cucinotta

(Survey-Tutorial Paper)

***Abstract*—Conventional cloud services and infrastructures are mainly designed to maximize utilization of resources and provide best-effort Quality-of-Service levels. However, many emerging use cases in both public and private cloud computing scenarios are time-critical in nature. For example, automated vehicles, smart cities, and automated factories, are all application domains characterized by the need for highly reliable and consistent low-latency services. The incorporation of predictable execution properties in cloud solutions is essential to meet these requirements. This paper provides an overview of the current research landscape in cloud computing, summarizing the key aspects to enable support of time-critical applications. The paper explores various levels of the typical cloud software stack: machine virtualization and containers, resource management and orchestration, fault tolerance, serverless computing, data storage and management, and communications.**

***Index Terms*—Predictable cloud computing, real-time virtualization, resource management and orchestration, real-time cloud storage, fault tolerance, deterministic networking.**

I. Introduction

SINCE the initial long-dreamed vision of computing as utility [1], cloud computing has become a disruptive technology with a steady ever-increasing impact on many sectors and businesses. The major characterizing aspects of cloud services is the illusion of infinite computing resources available on demand, and the ability to deploy horizontally scalable applications [2], [3]. These features relieve users of the burden of managing *physical infrastructure*. More recently [4], it became clear that cloud users are now facing a proliferation of *virtual resources* to be managed. This led to the widespread adoption of cloud-native applications developed using Platform-as-a-Service (PaaS) solutions, in addition to Infrastructure-as-a-Service (IaaS) ones. The latest evolution is serverless computing [5], [6], [7], where developers focus solely on writing code, leaving server provisioning and administration to the cloud provider. The relentless growth of cloud computing has been accompanied by a steep evolution of *networking* technologies in terms of mobility and reliability.

Received 17 March 2024; revised 19 December 2024; accepted 30 January 2025. Date of publication 5 February 2025; date of current version 10 April 2025. *(Corresponding author: T. Cucinotta.)*

R. Andreoli, L. Abeni, and T. Cucinotta are with Scuola Superiore Sant'Anna, 56127 Pisa, Italy (e-mail: remo.andreoli@santannapisa.it; Luca.Abeni@santannapisa.it; tommaso.cucinotta@santannapisa.it).

R. Mini, P. Skarin, H. Gustafsson, and J. Harmatos are with Ericsson Research, 16483 Lund, Sweden (e-mail: raquel.mini@ericsson.com; harald.gustafsson@ericsson.com; janos.harmatos@ericsson.com).

Digital Object Identifier 10.1109/TSC.2025.3539197

TABLE I
Time-critical Use Case requirements [9]

Domain	Use cases	Bw.	Lat.	Rel.	Adp.
Industrial control	Process monitoring	L	L	H	N
	Control to control in production line	L	L	H	N
	Programmable Logic Controller for robots	L	L	H	N
	Smart grid control	L	L	H	N
	Machine vision for robotics	H	L	H	N
	Closed-loop process control	L	UL	H	N
	Motion control	L	UL	H	N
Mobility automation	Automated container transport in port	L	L	H	N
	Machine vision for intersection safety	H	L	H	N
	Cooperative maneuvering of vehicles	L	L	H	N
	Collaborative mobile robots	L	UL	H	N
	Motion control of autonomous vehicles	L	UL	H	N
Remote control	Remote control with video/audio	H	L	O	Y
	Remote control with AR overlay	H	UL	O	Y
	Remote control with haptic feedback	H	UL	O	Y
Real-time media	Cloud-assisted basic AR	M	L	O	Y
	Premium-experience cloud-assisted AR	M	L	O	Y
	Cloud gaming	H	L	O	Y
	Interactive VR cloud gaming	H	L	O	Y
	Cloud-rendered AR	H	UL	O	Y
	Media production	H	UL	O	N

Acronyms: Virtual Reality (VR), Augmented Reality (AR); Network Bandwidth (Bw.), Latency (Lat.), Reliability (Rel.), Adaptive (Adp.); Ultra-Low (UL), Low (L), Medium (M), High (H), Optional (O).

These solutions have driven the novel trend of integrating cyber-physical-systems (CPSs) and cloud infrastructures to realize the fourth stage of the industrial revolution [8], a.k.a., "Industry 4.0". Interconnected CPSs, often being mobile, Internet-of-Things (IoT) or even wearable devices, equipped with wireless and low latency networking capabilities, unlock unprecedented opportunities to develop intelligent environments: smart cities, smart healthcare and smart transportation, often enhanced with virtual and augmented reality delivered through live-streaming services [9], [10]. These use cases are all *time-critical* in nature.

Unlike conventional general-purpose use cases, the correctness of a time-critical system depends not only on the results of the computation but also on the time at which the results are produced. Time-critical systems are therefore relevant for scenarios where the inability to meet the given latency target (the so-called *deadline*) would compromise users or critical system characteristics. What distinguishes these emerging use cases is the strictness of non-functional requirements, primarily network bandwidth, latency, and reliability. Table I summarizes the four major time-critical application domains and their related emerging use cases, as identified in [9], [10], [11]: industrial control, mobility automation and robotics [12], remote control,

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

and real-time media. For each domain, the use cases become more latency and timing critical, from top to bottom in the table, ranging from tens of milliseconds latency (i.e., low latency) to 1 ms (i.e., ultra-low latency). Reliability requirements can vary from 99% to 99.9999% for the most critical cases, where failure to comply with time-critical requirements may lead to catastrophic consequences (i.e. fatal accident to human personnel, damage to property, etc.). For example, in the industrial domain, a 5G/6G-enabled motion control system, which is responsible for moving and/or rotating parts of machines, requires stringent non-functional requirements, e.g., latency smaller than 2 ms and reliability greater than 99.999% [13], [14]. In the railway industry, time-critical systems controlling train movement and safety demand an end-to-end (E2E) latency smaller than 500 ms and reliability of 99.9999% [15]. Conversely, a cloud gaming service does not demand such stringent reliability, but it requires substantial network bandwidth, reaching up to 30 Mbps [10], and latencies in the 10-30 ms range. Additionally, most of the use cases in remote control and real-time media can adapt their network bandwidth demand based on transient conditions.

The adoption of cloud technologies for time-critical services is hampered by performance uncertainties: general-purpose cloud-based services are mainly designed to maximize throughput and utilization of shared resources, negatively affecting predictability at run-time and resulting in best-effort glsqos levels [16]. Predictability is essential to meet the requirements of the use cases mentioned above, so the ability for cloud platforms to meet stringent real-time and availability requirements is becoming increasingly important. This need has pushed computation and storage from centralized data centers towards the "edge" of the network. *Edge computing* is a distributed computing paradigm that brings computation and data storage closer to the source of the data [17], [18]. Since logical network proximity is entirely characterized by low latency and low jitter, edge computing reduces communication delays as well as the size of payloads transferred between the Internet and public/private data centers.

Combining cloud computing and time-critical systems is a complex problem that requires tackling not only network-related challenges but also those related to efficient (and predictable) access to the physical platform, as well as resource management techniques that consider time-criticality [19]. Considering what has been discussed so far, we state that a truly time-critical cloud platform requires the employment of highly reliable and latency-predictable networking, computing, and storage technologies, coupled with time-aware, fault-tolerant resource management mechanisms. Until all these requirements are jointly met, no cloud provider will be able to take responsibility for establishing a proper glssla for stable and predictable levels of service. This article provides a survey of the research literature on cloud solutions enabling deployment of time-critical use cases and intends to answer the following questions:

1) What are the current challenges associated with using cloud technologies for time-critical applications?
2) What cloud solutions include mechanisms for high reliability and time-predictability?

Contrary to most previous surveys [19], [20], [21], [22], [23], [24], [25], we answer these questions by investigating the entire cloud architecture software stack. We perform a broad review of papers, utilizing a breadth-first approach to cover several aspects and challenges intrinsic to cloud systems, that are critical to support time-predictable services. Fig. 1 presents a typical cloud computing architecture in which the physical components are represented at the bottom of the figure. As we go from the bottom to the top, we increase the abstraction level all the way up to the applications. The management activities, responsible for the coordination among the cloud resources, are represented by the vertical box on the right. In this paper, we cover all technical aspects presented in Fig. 1, and the colors used in this figure represent the way we divided the topics in this survey: Compute (green), Orchestration (orange), Storage (blue) and Communication (red). The rest of this paper is organized as follows: Section II describes the evolution of machine virtualization techniques, showing how their performance and support for time-critical applications improved over time. In Section III, we present orchestration techniques in cloud infrastructures. We cover topics from resource management to fault tolerance and serverless computing. Section IV presents an overview of storage technologies with latency control capabilities and performance guarantees for time-critical applications. Section V is dedicated to time-critical communication techniques. The end of each section includes a discussion on the surveyed state-of-art and challenges in achieving time-predictability. Finally, Section VI draws conclusions on the current development state of a holistic cloud-based system for time-critical applications.

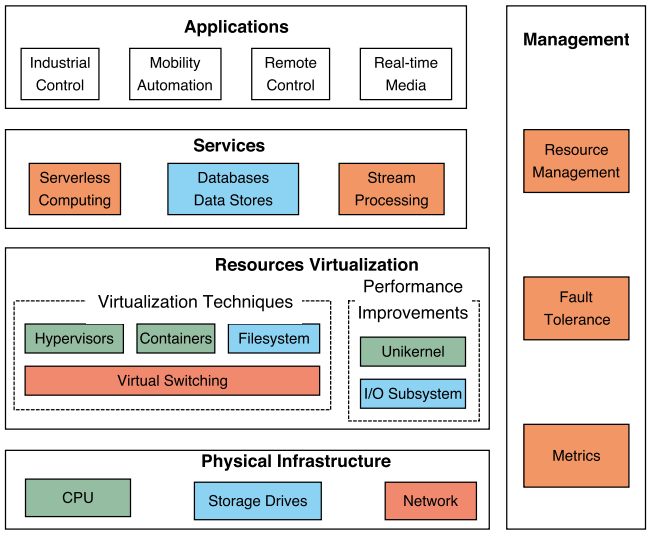


Fig. 1. Cloud computing architecture with colors representing the taxonomy used in this survey: Compute (green), Orchestration (orange), Storage (blue), and Communication (red).

II. Compute

This section discusses key challenges and proposed solutions for virtualization of the execution environment when executing time-critical applications. General-purpose machine virtualization mainly focuses on spatial protection and isolation (for example, the memory or files used by an application should not be accessible by others). However, for time-critical applications, we need also *temporal* isolation, i.e., the ability of an application

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF THE KEY CHARACTERISTICS OF THE REVIEWED PAPERS IN THE "COMPUTE" DOMAIN

Paper(s)	Virtualization Mech.	Main Contributions	Timing Properties & Guarantees	Benchmarks
[27]	-	RT Scheduling of SOA and Java tasks, no SMP	Temporal Isolation w/CBS	Synth, Real
[28]	-	RT Scheduling of web services, no SMP	Temporal Isolation w/CBS	Synth
[29]	L4 μ−kernel	Cache Coloring	Temporal Isolation	Synth
[30]	Xen	Modified Credit Scheduler	Reduced Latency	Synth
[31]	Xen	Multiple fixed-priority schedulers	Schedulability Guarantees w/CSF	Synth
[32]	KVM/HVM, Docker	Experimental Comparison	-	Synth
[33]	KVM	RT Scheduling of VMs	Temporal Isolation w/CBS	Synth
[34]	KVM	SCHED_DEADL. on KVM vCPUs	Temporal Isolation w/HCBS, Analysis w/CSF	Synth
[35], [36]	KVM, Xen	Experimental comparison	Reduced latency	Synth
[37]	Firecracker	KVM w/o QEmu	Reduced latency and boot time	Synth, Real
[38]	Docker	Perf. Isolation for containerized storage services	Reduced Latency, Increased Throughput	Synth
[39]	Docker	Flexible Deferrable Server	CPU Reclaiming for Non-RT	Synth
[40], [41]	LXC, OpenStack	Hierarchical RT Scheduling	Temporal Isolation w/CSF	Synth
[42]	Docker, K8S	RANs RT Cloud with PREEMPT_RT and DPDK	Reduced Latency	Real
[43]	Docker	FP Scheduling w/RTAI	Reduced Latency, No temporal Isolation	-
[44], [45]	Docker	FP/EDF Scheduling w/RTAI	Temporal Isolation	Synth
[46]	Xenomai, Docker	Xenomai w/PREEMPT_RT Linux containers	Reduced Latency	Synth
[47]	Xenomai	RT scheduling of containers with RT networking	Temporal Isolation w/co-kernel	Synth
[48]	Unikernels	UKs for NFV	Better Latency/Throughput	Synth
[49]	Unikernels	UKs for Serverless Comp.	Reduced Latency	Synth
[50]	Unikernels	Deferrable Server for UKs	Timing Guarantees via CSF	Synth

Acronyms: Real-Time (RT), Radio Access Network (RAN), Network Function Virtualization (NFV), Fixed Priority (FP), Earliest Deadline First (EDF), Constant Bandwidth Server (CBS), Kubernetes (K8S), Symmetric multiprocessing (SMP), Unikernels (UKs), Synthetic/Realistic workload (Synth/Real).

to respect its temporal constraints should not depend on the other applications that do not interact with it. To do this, a virtualization system has to employ *appropriate scheduling and resource allocation* to provide *low and predictable latency* [26]. These requirements are discussed with a focus on hypervisor-based Virtual Machines (VMs), containers, and unikernels, in the next sections. The main characteristics of the various papers are summarized in Table II.

A. Virtual Machines

Traditional hypervisors can provide a limited form of *temporal isolation* among hosted execution environments with their ability to partition the available physical resources, e.g., via core pinning and no over-subscription of CPU, memory and storage. Strong temporal isolation with a finer-grained allocation of resources is achievable only by using specialized solutions that combine real-time (RT) kernels and hypervisors, appropriate resource scheduling, and cache partitioning techniques, like cache coloring [29] or Intel's Cache Allocation Technology.[1] However, these are used mostly in safety-critical RT embedded systems [19]. For other domains, a few solutions approximate the concept enriching general-purpose OSes and hypervisors with various mechanisms.

For example, when using KVM on Linux with more VMs sharing the same physical core(s), the bandwidth control[2] mechanism allows for controlling the maximum fraction of glscpu time consumed by the virtual CPUs (vCPUs) of each VMs, but there is no guarantee on the minimum fraction of received CPU time, or on the timing of the allocation. As a result, this mechanism cannot guarantee the respect of tight temporal constraints for the virtualized applications. In [30], the Xen credit (proportional-share) scheduler has been improved for soft RT workloads, and used for reducing the response times of a glssip server, albeit it is unable to provide formal temporal guarantees. In RT-Xen [31], a theoretical-founded RT scheduler has been added to Xen, allowing to perform a formal schedulability analysis of the guests' RT tasks based on the glscsf [51], [52]. This algorithm, based on the glsedf and deferrable server algorithms, is now integrated into the mainline Xen under the name of glsrtds. It has also been integrated in OpenStack (see Section III-A). This is a step forward to support RT applications, but a mechanism to dimension the scheduling parameters is still missing [53]. RT scheduling of a glsvm vCentral Processing Units (CPUs) can be used also with type II hypervisors such as glskvm, using a RT scheduler in the host kernel. For example, the glsirmos RT scheduler [33] implements the glscbs algorithm [54], based on glsedf, for groups of threads. A related old line of works include *real-time service-oriented architectures*, for example as realized in the RI-MACS [28] and Llama [27] projects, where similar RT CPU schedulers were used to serve distributed workloads in a factory automation context. However, most of these older works belong to the pre-Cloud era, so they were not considering several Cloud-specific aspects, like virtualization, containers, orchestration, load-balancing, or even multi-processor architectures.

As of today, the glscbs algorithm is available in the mainline Linux kernel as the new `SCHED_DEADLINE` RT scheduling policy.[3] This allows for using hierarchical RT scheduling for the glskvm vglscpu threads without patching the host kernel [34]. Besides RT CPU schedulers for VMs, other mechanisms to provide low latency have also been investigated. In [35], the latency introduced by Xen and glskvm is evaluated, showing that

[1] https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-cache-allocation-technology.html

[2] https://docs.kernel.org/scheduler/sched-bwc.html

[3] https://www.kernel.org/doc/Documentation/scheduler/sched-deadline.txt.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

glskvm can provide RT performance comparable with the one experienced by Real-Time Operating Systems (RTOSs) running on bare metal, while Xen needs some fixes [36]. Some other hypervisors, such as ACRN [55] and Jailhouse [56], have been developed from scratch focusing on latency reduction. In many cases, this goal is achieved by dedicating one entire physical glscpu core to each vCPU, avoiding any kind of scheduling. This approach is also used by separation kernels [57] or partitioning hypervisors which are commonly used in embedded systems (see for example Bao [58], Xtratum [59], or the SEL4 microkernel [60]). However, these hypervisors are mostly used in embedded hard real-time use cases or simple edge real-time computing scenarios [61]. They support a limited number of hardware architectures typically employeed in specific application domains, like automotive, for which the software stack often needs to be developed according to expensive certification processes. Overall, these are more focused on hosting simple RT controllers, rather than supporting a full-featured Linux OS, for example, thus they're not usable in general cloud computing infrastructures.

Some works aim at improving the start-up time of VMs for elastic and serverless workloads. For example, the FireCracker [37] "micro-VM" is a hypervisor still based on the glskvm kernel module but dropping its dependency on glsqemu. This results in limited features compared to traditional Virtual Machine Monitors (VMMs), but the solution is ideal for services such as AWS Lambda that need to start many containers/VMs in a short time, even with some loss in disk throughput and I/O latency [37].

B. Containers

Containers can provide better performance than VMs, with the ability to even reach bare-metal levels, but are traditionally considered less secure, due to the need for sharing the host machine kernel. They also suffer of a weaker form of isolation: for example, in [38], the lack of I/O performance isolation on disk access for docker containers is highlighted, and an optimization of the I/O concurrency level is proposed to control the performance of each container.

Similarly to what stated above on VMs, containers allow for a very limited form of temporal isolation by employing partitioning of the physical resources, and limited CPU scheduling abilities. The latter are usually provided through the bandwidth control mechanism mentioned above, supported by most of the current container managers [39]. For example, tuning the Linux CPU scheduler for improving the responsiveness of real-time packet processing in Cloud-RAN scenarios has been proposed in [62]. However, to provide real-time guarantees to applications, a formal schedulability analysis is needed, like the glscsf [51], [52] mentioned in Section II-A. Indeed, implementations of glscsf have been realized not only for hypervisors [31], [34], but also for containers. For example, the Hierarchical CBS (HCBS) has been proposed in [40], an extension of the `SCHED_DEADLINE` glscpu scheduler in the Linux kernel. glshcbs allows to attach groups of tasks to a multi-processor glscbs, where individual tasks are scheduled using the standard priority-based RT scheduler within the time-slices allocated to the group. These techniques can also be used to provide guarantees to complex applications, with interacting tasks and more flexible performance guarantees based on probabilistic analysis [41].

Other works investigated how to reduce the latency of containerized applications (see [23] for a survey). This can be done by using a preemptible host kernel or by using a co-kernel approach. The authors in [42] evaluate the latency introduced by Docker containers with various kinds of kernel (standard, low latency and fully preemptible — Preempt-RT), showing the advantages of using Preempt-RT and DPDK for low-latency networking. In RT-CASES [43], the RTAI co-kernel was used to provide containers with low scheduling latency. However, the first proposed mechanism supported only fixed-priority scheduling, so there was no temporal protection among RT threads. The approach was extended in [44], adding a monitoring component to cope with RT threads trying to overrun their known WCET, still using the fixed-priority scheduler available in RTAI. However, monitoring was based on a sampling approach, which implies either low precision or high overheads, with authors suggesting dedicating a CPU just for the monitoring thread. Finally, glsedf scheduling with hard CBS was added in [45], modifying the RTAI scheduler.

Another work proposing a co-kernel can be found in [46], where low latency in containers is achieved by combining Xenomai and Linux with a Preempt-RT kernel. In [32], Preempt-RT, Xenomai and some instances of the AWS EC2 cloud using the HVM hypervisor are compared in terms of latency measured with cyclictest. Authors claim that some HVM-based instances can be suitable for migrating time-critical industrial control applications from bare metal to IaaS services, using a Preempt-RT Linux kernel, but they neglect the implied impact of network delays in their evaluation. Another interesting work can be found in [47], where Xenomai is extended with a hierarchical reservation-based CPU scheduler, called SCHED_DS, to obtain real-time scheduling of containers. Basically, thread groups from different containers are scheduled using a deferrable server algorithm, then within each container threads are scheduled by priority. The solution relies on the Xenomai RTnet [63] module to add also TDMA-based real-time networking guarantees on Ethernet networks.

C. Unikernels

VMs are often preferred to containers thanks to their improved isolation. However, they bring higher overheads due to the higher complexity of the overall software stack on the node, resulting in higher latency/execution times, boot times, and memory footprint. Unikernels aim to mitigate these drawbacks, reducing the complexity of a glsvm software stack. Indeed, a unikernel is a library glsos directly linked with the target application, and deployed in a glsvm; see [64] for a survey with detailed information on unikernels and containerization solutions for edge computing. Various papers investigated on performance improvements and overhead reductions for unikernels. In [48], unikernels are proposed to get the performance of containers with the security of VMs, with experiments performed with

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

IncludeOS.[4] In [49], a serverless platform is proposed based on unikernels (SOLO5), comparing the achieved performance with the one of Apache OpenWhisk,[5] based on Docker containers.

Some studies [65] show that unikernels can even provide lower latency than Docker-based containers in some situations. However, these performance improvements depend on the unikernel used for the experiments: a performance comparison [66] carried out using the glsrumprun[6] unikernel shows a decrease in memory and glscpu usage compared to glskvm, but also increased latency and lower throughput. Finally, the work in [67] presents a more systematic benchmarking of the various technologies that can implement a container (hypervisors, kernel virtualization, micro-VMs, intercepting system calls, unikernels,...) showing that the combination of micro-VMs and unikernels allows to greatly reduce the overhead introduced by hypervisor-based virtualization.

Unfortunately, most of the research on unikernels did not consider the requirements of time-critical applications. The only notable exception is the work presented in [50], which shows how unikernels allow for better timing guarantees for single-threaded applications, using an optimized version of the traditional glscsf analysis, which is anyway quite pessimistic.

D. Discussion

Summing up, in the compute area, there has been significant academic research on virtualization techniques to provide real-time glscpu scheduling. Some of these works produced mechanisms that have been integrated into mainline glsos kernels and hypervisors. As a result, two of the most commonly used open-source hypervisors (glskvm and Xen) are suitable for being used in time-critical use cases, providing isolation, low latency, and predictable real-time glscpu scheduling. On the other hand, these features are available for containerized time-critical workloads only applying out-of-tree patches. Various works focused on the advantages of using unikernels, in terms of boot-time reduction for example. However, this technology does not seem mature enough to be used in production environments and the impact of unikernels on time-critical applications has not been fully evaluated. There are also numerous other operating system and hypervisor solutions in the embedded and real-time systems domain, addressing real-time glscpu scheduling, including also commercial products such as VxWorks,[7] Helix,[8] Erika Enterprise,[9] PikeOS[10] or others, mostly providing certified solutions for specific application domains, but often lacking portability on general-purpose commodity hardware, as needed in cloud computing. Even though the basic technologies to provide predictable real-time glscpu scheduling already exist, there is no mechanism to properly dimension the scheduling parameters based on the application requirements. In addition, glscpu scheduling has not been integrated with other resources such as network and storage in order to support time-critical applications that are not purely glscpu-intensive.

[4]More information is available at: https://www.includeos.org/.
[5]More information is available at: https://openwhisk.apache.org/.
[6]For more information, see: https://github.com/rumpkernel/rumprun.
[7]More information at: https://www.windriver.com/products/vxworks.
[8]More information at: https://www.windriver.com/products/helix.
[9]More information at: https://www.erika-enterprise.com/.
[10]More information at: https://www.sysgo.com/pikeos.

III. Orchestration

Performance-aware cloud orchestration has been studied at large, encompassing several challenges related to time-criticality, like: efficient resource scheduling and allocation with isolation among virtualized tasks; fault tolerance; optimized serverless architectures; and, RT data streaming. For instance, the Kubernetes scheduler has been enhanced by leveraging system-level metrics [68] to reduce interference, or using spare backups and ping-based latency-awareness [69] for reliability and low-latency. However, for time-critical workloads, we need proper mechanisms ensuring the end-to-end latency is not disrupted by transient problems nor faults. Moreover, most of the papers in this area consider tasks with relatively long deadlines, as found in big-data or scientific workloads, but some papers covering low latency in the micro/milli-second range exist as well. Table III summarizes key aspects of the works on orchestration discussed below.

A. Task Resource Scheduling & Admission

To provide stable performance, cloud services use extensively load balancers to spread the load across a pool of instances, and horizontal elasticity to scale the pool so to match target performance levels. A number of works exist [89], [90] surveying key approaches to load-balancing and elasticity in cloud computing. However, most of the elasticity approaches are merely *reactive*, i.e., they correct the system behavior once it deviates from the desired state, meaning that they do not include effective mechanisms for *temporal isolation*.

RT workloads may be hosted in existing cloud management solutions by leveraging on resource partitioning to obtain similar performance as with a dedicated physical machine (as supported by AWS or GCP bare-metal instances, or the Ironic[11] service in OpenStack, for example). This gets rid of temporal interferences among instances, but it also cancels several advantages in adopting cloud infrastructures. When switching to regular instances that share physical servers, many instance types offered by cloud providers offer a 1:1 mapping between virtual cores with physical ones. This is also supported in OpenStack,[12] which also allows to configure a pre-emptable real-time kernel, tune the glsdvfs and disable deep idle states on the host to reduce response latencies. However, none of these solutions support the use of RT scheduling policies (i.e., glscbs) to ensure proper temporal isolation among virtual cores when sharing physical cores.

To overcome these limitations, the glsirmos project [33] proposed a cloud management solution for interactive real-time and multimedia distributed applications. It included the glsisoni [91], enabling deployment of complex distributed real-time software components in the form of Virtual Service Networks (VSNs),

[11]More information at: https://ironicbaremetal.org
[12]See: https://docs.openstack.org/nova/wallaby/admin/real-time.html

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SUMMARY OF THE KEY CHARACTERISTICS OF THE REVIEWED PAPERS IN THE "ORCHESTRATION" DOMAIN

Domain	Papers	Main Contribution	Timing Property & Guarantees	Benchmark
Resource Scheduling & Admission	[41]	Control the interferences of co-located real-time services	Temporal isolation	Synth, Real
	[70]	Co-hosting of RT and non-RT VMs on OpenStack	Schedulability experimental evaluation	Synth
	[?], [71], [72], [73]	RT scheduling on K8S	Temporal isolation w/HCBS	Synth
	[74]	RT scheduling on K8S + ROS	Temporal isolation w/SCHED_DEADLINE	Synth, Real
	[75]	Criticality-aware scheduling in K8S	Temporal isolation	Synth
	[76]	Time-slice control scheme for VMs	Reduced exec. time	Real
	[77]	Energy-efficient RT scheduling in Cloud	High deadline guarantee ratio	Synth, Real
	[78]	Response time analysis for aperiodic time-critical services	-	Synth, Real
Fault Tolerance	[79]	Low latency extension of Raft	Reduced tail latency	Synth, Real
	[80]	High-performance consensus protocol	Reduced latency	Synth, Real
	[81]	RDMA-based consensus protocol	Reduced repl. latency	Synth
Serverless	[82]	SLO-aware OpenFaaS extension	Function invocation rate guarantees	Synth
	[83], [84]	partitioned-EDF scheduling for RT functions	Schedulability experimental evaluation	Synth
Real-time Stream Processing	[85]	Timely dataflow system	Reduced latency, increased throughput	Synth, Real
	[86]	Congestion-aware scheduler in Apache Storm	Reduced latency, increased throughput	Real
	[87]	Locality-aware resource allocation, co-flow scheduling	Reduced latency, increased throughput	Real
	[88]	RT Apache Storm	Temporal Isolation	Real

Acronyms: Real-Time Virtual Machine (RT VM), Best-Effort Virtual Machine (non-RT VM), Synthetic workload (Synth), Realistic workload (Real), Kubernetes (K8S), Remote Direct Memory Access (RDMA).

i.e., Directed Acyclic Graphs (DAGs) of VMs, with attached precise end-to-end temporal constraints, expressed in either deterministic or probabilistic terms. These constraints could be respected thanks to the use of the above mentioned IRMOS real-time glscpu scheduler based on the glscbs, standard packet schedulers supporting glsqos guarantees, and proper storage management, as well as optimal glsvm placement [92]. More recently [41], a modified Nova component in OpenStack has been proposed: on the compute side, time-critical VMs are deployed by setting the required RT scheduling parameters on the underlying glshcbs RT scheduler; on the controller side, it embeds a placement logic that is aware of the RT computational bandwidth allocated to each time-critical instance. The solution is validated through extensive experimentation, where multiple co-located benchmarks from the domains of glsnfv and glsims are deployed in a real test bed. The results show that the proposed mechanism ensures temporal isolation of individual containerized activities. Similarly, RT-OpenStack [70] is a modified version of OpenStack for co-hosting time-critical and regular VMs, by: 1) integration of the RT-Xen hypervisor within OpenStack through a RT resource interface; 2) a RT glsvm scheduler to enable regular VMs to share hosts with RT VMs without temporal interference; and 3) a VM-to-host mapping strategy to provision RT performance to RT VMs while allowing effective resource sharing with regular VMs. In practice, RT-OpenStack performs RT admission control in the OpenStack compute scheduler. Then, RT VMs are scheduled using global glsedf.

Kubernetes has also been modified with real-time scheduling features, to host time-critical containers. RT-Kube [74] integrates Kubernetes directly with the SCHED_DEADLINE scheduler available in the mainline Linux kernel to support scheduling of RT ROS tasks and management of the CPU bandwidth. The authors provide experimental results based on synthetic benchmarks, and a real robot control use-case in a pick-and-place scenario. However, the proposal relies merely on the default configuration using Global EDF and an admission test which is only necessary, not sufficient to guarantee schedulability of RT tasks on multi-processors. Moreover, the paper does not discuss real-time networking, nor how they reconciled SCHED_DEADLINE, which deal with single-threaded processes only, and the typically multi-threaded architecture of ROS applications. RT-Kubernetes [71] addresses this issue by using the glshcbs extension to SCHED_DEADLINE, which allows setting a different runtime on each core/glscpu to avoid resource waste on many-core hosts. This contribution is also coupled with an admission test for finding proper nodes according to the available real-time CPU bandwidth. A quite similar approach can be found in [72], [73], which is also based on the same glshcbs real-time scheduler. Compared to RT-Kubernetes, the authors implemented a feedback loop that continuously monitor the resource usage and glsqos experienced by the containerized workload through a set of metrics (e.g., deadline misses, lateness, response time), so to consider this information in future scheduling decisions. Another paper exploring similar aspects with real-time task group scheduling can be found in [75]: the main differences lie in details of the admission test, placement strategy, and monitoring mechanisms.

Adaptive time-slice control (ATC) [76] is a solution to detect communication and computation phases in applications based on lock latency and cache misses. This information is used to shorten time-slices during communication phases and prolong them during computation phases. The authors focus on multithreaded and distributed applications including multiple interacting VMs. The method chooses longer time slices during compute phases to keep the cache filled. The evaluation is done for applications with clear communication and compute phases like distributed compilation.

In [77], glsprs is proposed for VMs with uncertain execution times and aperiodic real-time tasks, considering energy efficiency i.e., minimizing the active hosts while guaranteeing timing requirements. The scheduler is based on the task laxity, accounting for both the task durations and deadlines. In [78], cloud services with probabilistic latency guarantees are

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

presented, with a stochastic response time analysis for aperiodic services following a Poisson arrival process on computing platforms that schedule time-critical services as deferrable servers.

B. Fault Tolerance

We can find both proactive and reactive approaches to fault tolerance in cloud computing [21], [93]. Among the former ones, there are preemptive migration and rejuvenation, which reduce the likelihood of a fault occurring by moving the task away from the potentially faulty system in advance, or by making sure to repair the system before the fault occurs. Among the reactive methods, checkpointing and replication are common, which enable the possibility to replay tasks at another instance after a fault occurred. For real-time cloud applications, most of the reactive approaches are based on replication due to the relatively low latency recovery time. On the other hand, checkpointing requires at least a couple of seconds in downtime during state snapshot for common containerized application state sizes. Moreover, the cloud management and orchestrator itself may have a variety of failures, as highlighted in [94], where a Kubernetes fault injection framework is proposed, based on several reported observations from real deployments. The same authors provided also an interesting analysis [95] of the time needed to handle and repair detected faulty pods in Kubernetes. However, only a few papers deal with fault tolerance for cloud services with latency requirements in the millisecond range. This is interesting for some of the more demanding use cases like tight automation control systems. These are commonly handled today by dedicated embedded hardware, but they are expected to see much increased compute requirements, so that offloading to a cloud system will be beneficial [96], [97]. Both scalability and fault-tolerance can be handled with the same design principle of replication. However, for elastic services, too many replicas improve fault-tolerance at the cost of hurting the performance. This is due to the fact that consensus overhead grows with the number of nodes. In [79], it is explored if adding nodes can both increase performance and fault tolerance for stateful data-center applications that require fault-tolerance, low latency, and scalability. They present HovercRaft, a load balanced glsrpc protocol extended with a Raft derivative that separates the ordering and replication of requests. The replies are sent from any of the replicas, reducing the work for the leader replica and increasing scalability. The load balancing policies reduce the tail latency but provide no deadline guarantees.

Another interesting work can be found in [98], where a strategy is presented to reduce latency variations when using load balancing and request redundancy, i.e., *hedging*. Measurements of microsecond-level variations in request latency show that excessive redundancy may lead to congestion and increased latency variations, whilst redundancy permits using the earliest response, achieving a more consistent latency.

Nezha [80] is a high-performance and deployable consensus protocol that exploits accurate software clock synchronization. It does not require special hardware or physical network access, making it easily deployable in virtualized environments. Instead, it uses a new primitive to send requests to several replicas by multicast which orders client-to-replica requests by deadlines. The deadlines are specified in the synchronized wall clock time. Performance is evaluated for two applications, Redis and a prototype financial exchange CloudEx, which show fault tolerance is achieved with only a modest performance degradation.

Another low latency fault tolerant replication scheme using leader consensus is based on glsrdma [81], introducing a new glssmr protocol, called Mu, that carefully leverages glsrdma to lower the fault detection time. Mu uses a conceptually different method based on a pull-score mechanism over glsrdma. The leader increments a heartbeat counter in its local memory, while other replicas use glsrdma to periodically read such counter. A badness score is calculated based on the number of successive reads that have returned the same value. Replicas declare a failure if the score is above a badness threshold, corresponding to causing a timeout. Unlike traditional heartbeat signals, this method can use an aggressively small timeout without false positives because network delays slow down the reads rather than the heartbeats. This way, Mu detects failures usually within 600 microseconds. When a failure occurs, this scheme is shown to take less than a millisecond to recover, whereas HovercRaft takes 10 milliseconds [79].

Fault detection is a related research field dealing with observability challenges both at the infrastructure and application levels. In time-critical services, faults manifest not only as conventional infrastructural problems (i.e., network issues or hardware failures) but also as deadline misses. However, only a few works deal with low latency fault detection for real-time cloud systems. For example, in [99], [100], [101], a low-latency detection system is proposed, alongside a performance model that makes use of timely fault detection to evaluate the feasibility of time-constrained, cloud-native applications.

C. Serverless Computing

The serverless paradigm provides event-driven services that relieve developers from the burden of server provisioning and network infrastructure management. In glsfaas, developers deploy functions i.e., isolated and stateless components processing inputs to obtain outputs, using containers in a cloud environment (e.g., a Kubernetes cluster). These are created, scaled and shut down on-demand, according to the instantaneous workload. This structure is very useful in event-driven non-critical glsiot but tends to exhibit large latency and jitter. One of the issues with FaaS is the “cold” start-up time, incurred when an instance is not available yet, on a function invocation. Another criticality is due to the fact that often applications offload several parts of their computations to a chain of function invocations. To create stateful applications, cloud storage services are also needed.

Existing serverless platforms do not support real-time applications in the deterministic sense. An extension to OpenFaaS has been proposed [82] for glsslo guarantees in terms of functions’ invocation rate. More specifically, an admission controller accepts the deployment of functions based on the system state and

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

requested invocation rate guarantees, and a predictive manager provisions the underlying containers accordingly.

RT-FaaS has been proposed [83], [84] for services that can execute a mix of real-time and non real-time functions, providing the former with response time guarantees. A heuristic spatial allocator is used to minimize the processors allocated to RT tasks. The framework assumes real-time communications ensuring known timing of data transfers in remote service invocations, and knowledge of the worst-case processing times. On the hosts that execute the functions, a per-function glsedf scheduler is used to queue and serve the requests.

D. Real-time Stream Processing

Many time-critical applications require low latency interactive access to results of data stream processing pipelines, such as real-time analytics of data coming from IoT or wearable devices. In this context, the potential value of the extracted information rapidly decreases over time, hence the need for stream processing platforms designed for high responsiveness. One of the first works in this context is Naiad [85], a general-purpose low-level programming abstraction for executing distributed data-parallel, cyclic dataflow programs with mixed high throughput and low latency requirements. It implements a novel computational model which enriches dataflow computations with logical timestamps to track the global progress of the computation, together with an efficient notification-based coordination protocol that allows for several communication policies, such as prioritizing the delivery of notifications with the earliest timestamp to reduce end-to-end latency.

Most of the existing processing frameworks have been designed under the assumption of an abundance of memory and powerful computing resources, resulting in inefficient resource usage and unpredictable I/O costs. In the context of time-criticality for stream processing engines, this is tackled by focusing on resource-constrained edge computing scenarios, and on the idea of processing data streams "close to the data source". EdgeWise [86] modifies Apache Storm[13] to improve both throughput and latency on Edge-based Storm deployments. It introduces a fixed-size worker pool execution model, replacing the one-worker per-operation architecture of Storm, and a congestion-aware user-level scheduler that ensures I/O queue lengths do not exceed available memory by assigning a ready thread to the operation with the most pending data. Both changes reduce the intrinsic non deterministic nature of general-purpose OS scheduling, making stream processing more amenable for memory-constrained edge environments. Amnis [87] is a novel stream query processing engine that extends Apache Storm to tackle the challenges in edge computing scenarios. Amnis optimizes the throughput and end-to-end latency of data streams by: employing a data locality-aware resource allocator for query operators; adopting a load-aware scheduler that considers the dynamically varying load conditions and resource requirements of each operator; and considering the dependency between data flows to be transferred through the network so that the volume of data can be controlled with a rate limiter. Finally, Storm-RTS [88] replaces the default worker-based execution model of Apache Storm with a rate-based abstract machine, building a stream processing workflow as a chain of rate-guaranteed function invocations. This allows for flexible resource reconfigurations achieving predictable performance.

[13] https://storm.apache.org/

E. Discussion

In the orchestration research area, we noticed a great focus on best-effort cloud services or long duration workloads, where the goal is to ensure that the majority of users receive a satisfactory service. For example, it is probabilistically harmless to have temporal service disruptions due to a noisy neighbourhood problem, or during a scale-out operation or the replacement of a faulty instance. However, this is in contrast with the traditional real-time deterministic view of per-request latency guarantees, as needed by time-critical applications. Many researchers dealt with the potential of horizontal elasticity, with a shortage of efforts in enriching cloud management solutions with key features, such as: integration with strong temporal isolation features among instances sharing physical resources, as found at the hypervisor or OS levels in compute nodes (see Section II), with a fine per-request granularity control on the end-to-end latency, and low latency fault detection and recovery. Fault tolerance is a standard concept in cloud and distributed computing, but it is mainly implemented without worst-case latency guarantees. Only a few approaches tried to integrate cloud technologies with worst-case guarantees despite faults, as needed by time-critical applications. This confirms the need for further investigations in this direction.

IV. Storage

Modern cloud-native data-driven applications with time-critical requirements, such as multimedia streaming and cloud gaming platforms, need specialized storage services with latency control capabilities and performance guarantees. To this end, cloud providers must employ optimized software stacks for local storage nodes, as well as ad hoc database architectures equipped with specialized task and resource schedulers. Most of the literature does not employ solid real-time methods, focusing instead on adaptive latency control mechanisms for the latency-sensitive use case. In the following, the existing literature is summarized, with a focus on: i) local storage optimizations that employ low-level mechanisms to control latency, such as cutting-edge storage drives and filesystems optimized for virtualized environments; and, ii) the goals and challenges of traditional real-time databases, presenting the current solutions for the control and/or reduction of latency. A summary of the key characteristics of the papers reviewed in this section is provided in Table IV.

A. Local Storage Optimizations

Before the introduction of Solid State Drives (SSDs), a typical source of unpredictability was the high seek latency of traditional rotational disks, a problem tackled in two ways: using in-memory storage only, as proposed in [114], or by

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SUMMARY OF THE KEY CHARACTERISTICS OF THE REVIEWED PAPERS IN THE "STORAGE" DOMAIN

Paper(s)	Storage Context	Main Contributions	Timing Properties & Guarantees	Benchmarks
[102]	I/O scheduler	Reservation-based exclusive disk access	High throughput with fairness guarantees	Synth, Real
[103], [104]	SSD Subsystem	Storage interface exposing SSD internals to the host	I/O predictability and isolation	Synth
[105]	SSD Opt	Leverage SSD internals to reduce in-device R/W contention	Reduced tail latency	Synth
[106]	SSD Opt	Hybrid polling scheme for Ultra-low latency SSD	Reduced CPU utilization and I/O latency	Synth
[107]	I/O Stack Opt	Layer-aware I/O stack for containers to reduce host-level resource contention	Reduced Latency	Synth
[108]	Overlay FS	Block-level accessibility for containerized storage systems	Reduced average latency	Synth, Real
[109]	Container image	File sharing between images and on-demand file retrieval	Reduced storage space usage	Synth
[110]	RTDB	QoS-aware feedback control for bursty workloads	Bounded response times	Real
[111]	RTDB	QoS-aware feedback control for embedded storage	Minimize real-time transaction tardiness	Synth
[112], [113]	NoSQL	QoS and QoD aware task scheduling	Minimize requirement violations	-
[114]	In-memory NoSQL	Transparent, locality-optimizing data migration for ultra-low data access	Reduced latency	Synth
[115]	NoSQL	Priority-based performance differentiation	Reduced/stable latency for high-priority req.	Synth, Real
[116]	NoSQL	Multitiered data store with adaption to workload changes	Cost-vs-performance/latency tradeoff	Synth
[117], [118]	NoSQL	Fully-managed cloud service with differentiated performance	Guaranteed throughput and low latency	Real
[119]	NoSQL	Preserving ACID transactions keeping linear scalability	Reduced average latency	Synth
[120]	NoSQL	Fog and Context aware replica placement and consistency	Reduced latency	Synth
[121]	NoSQL	Autonomic rate limiter for latency-critical edge	Reduced latency	Synth
[122]	NoSQL	Mobility and Access pattern aware data placement	Reduced tail latency	Synth, Real

Acronyms: Real-Time Database (RTDB), NoSQL Database (NoSQL), File system (FS), Optimization (Opt), Synthetic/, Realistic workload (Synth/Real).

employing special disk access scheduling techniques coupled with pessimistic analysis, such as the glsbfq scheduler [102], which combines disk idling with timestamp-based scheduling to preserve both guarantees and high throughput for synchronous requests. BFQ reserves a fraction of the disk throughput and ensures exclusive disk access for a certain amount of budget time every period for each application. Although SSDs eliminated the high seek latency problem, a traditional block I/O SSD suffers from unpredictable performance due to the often non-disclosed sector allocation, prefetch, and caching logic within the drive controllers, as well as suboptimal resource utilization. Hence the introduction of open-channel SSDs for fine-grained, application-specific management of the data placement strategy as well as of the physical I/O scheduling. LightNVM [103] is a novel I/O subsystem within the Linux Kernel that exposes the parallelism capabilities and storage media characteristics of open-channel SSDs to the host OS. LightNVM provides an interface where application-specific abstractions can be implemented. A recent work that makes use of this interface is RAIL [105], a novel SSD management technique to reduce read tail latency in the presence of write operations. The authors state that since write buffer flushes can contend with reads on flash devices and bring long latency tails, RAIL redirects user and garbage collection writes towards independent physical flash chips, thus exploiting the internal parallelism of NVME SSDs in order to eliminate the possibility of reads being stalled by any high latency operation (read-after-write). RAIL effectively enforces predictable performance for Flash accesses, achieving 7× lower tail read latency than conventional SSD management techniques. Open-channel SSDs have been replaced by glszns SSDs [104], a standardized, media-agnostic interface among different SSD vendor implementations.

The recent introduction of ultra-low latency SSDs naturally shifted the direction of I/O performance improvements towards the I/O stack, which now accounts for a large fraction of the application-perceived I/O latency. An adaptive hybrid polling scheme has been proposed [106] that combines polling and interrupts. The proposal makes optimal sleep time decisions to maximize the I/O performance and minimize the CPU cycles used for polling. An optimized layer-aware I/O stack has been proposed in [107] to reduce file search overheads for containers and alleviate resource contention in the native file system. This is done in two steps: i) by employing a modified virtual file system that locates files based on layer information to enable simultaneous glscow operations, thus reducing file open and lock contention; and ii) by replacing the journal service of the native file system with multiple, isolated micro-journals per container, thus reducing resource conflicts caused by shared resources. Experimental results show improved latency and throughput for a set of containerized data-intensive applications.

Baoverlay [108] is a lightweight overlay filesystem that enables block-level access for improved write performance in container-based storage systems. It logically decomposes an overlay container image file into equally-sized blocks so that a data update originated by the container does not involve copying the whole file, thus reducing the write latency overhead of a traditional glscow mechanism, which could incur into long "read-and-then-write" I/O operations.

Gear [109] is a container image format for efficient image storage and deployment. It introduces an index structure decoupled from the regular image files, which allows for on-demand retrieval of necessary files only and file sharing between different images while keeping full compatibility with Docker. The proposal effectively saves up to 54% storage space in the registry but also accelerates container deployment under scenarios with limited bandwidth.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

B. Real-time Databases

Unlike a traditional database management system, a real-time one [123] employs several optimizations to minimize the number of missed deadlines by reducing to the minimum the unpredictability of transaction execution. For instance, conventional resource (i.e. CPU and I/O) scheduling algorithms are designed to ensure fairness between tasks, whereas a real-time one schedules the next transaction as a function of its criticality and the tightness of its deadline. Similarly, the conventional concurrency control protocols, used to ensure consistency between concurrent transactions, may lead to priority inversion or deadlocks. Hence the use of lock-free algorithms [124] integrating transaction priorities into the conflict resolution logic, such as the one proposed by Lindström [125]. Another challenge is to completely eliminate I/O accesses, due to the unpredictability of disk access delays, by putting data directly in memory and employing recovery protocols [126] in case of hardware failure. In the context of classic real-time database systems, there is a multitude of feedback real-time scheduling and utilization control algorithms, such as Chronos [110], which dynamically controls the desired service delay through admission control of incoming transactions in proportion to the response time error, effectively managing the amount of backlog in the database.

Similarly, QeDB [111] employs a feedback control scheme for data-intensive, real-time applications hosted on embedded systems with memory constraints. QeDB simultaneously manages both I/O and CPU resources to guarantee the desired trade-off between timeliness of transactions and data freshness. By monitoring the I/O and CPU tardiness, QeDB computes the buffer hit ratio adjustment to allocate the proper amount of resources in a robust and controlled manner.

The advent of cloud computing raised interest in cloud-hosted databases for modern large web-based workloads with time-critical requirements. This shift towards highly scalable infrastructures led to the shortcoming of relational databases in favor of NoSQL architectures, which sacrifice ACID transactions in favor of relaxed consistency models, to achieve higher availability and scalability [127]. Although there is little recent academic literature on real-time cloud-based data stores, there are many prototypes for latency-sensitive workloads that could be coupled with traditional real-time database principles to ensure predictable performance. AQUAS [112], [113] is a replacement scheduler for Apache Cassandra[14] that efficiently allocates replica nodes to minimize the penalties incurred in violating a set of glsqos and glsqod requirements. It estimates the execution time of operations via a simple linear regression model for writes, and by looking at the execution times of previous queries for reads. AQUAS offers various query scheduling policies, such as FIFO or glsedf.

DAL [114] is an in-memory data backend with low latency and high reliability. It works both as a data store and a publish-subscribe message broker. A DAL cluster is made of a collection of distributed servers queried by clients. When a server detects that an item is mostly accessed from a specific remote client, it proposes a data move closer to such client. When the same physical machine co-hosts a client and a server instance, data items that are dominantly used by a single client will be accessed locally after a short transient.

RT-MongoDB [115] is a modification to the popular MongoDB NoSQL data store that makes use of POSIX nice levels and of a priority-based synchronization logic to offer reduced per-user or per-request response times: higher priority requests/clients are able to be served earlier than lower priority ones by preempting or starving the latter for arbitrarily long time windows. More specifically, the work exploits the MongoDB per-client thread model, as well as a concurrency control mechanism, to implement prioritization of the threads corresponding to higher-priority clients/requests.

Anna [116] is a cloud-native, distributed key-value store that dynamically adapts to workload variations based on three high-level goals specified by the administrator: average latency, cost–performance tradeoff, and fault-tolerance. Anna employs a policy engine that monitors the workloads and responds to variations that may violate the objectives through horizontal scaling, vertical data tiering (i.e. moving "hot" data to faster memory, demoting rarely used data to cold storage), and selective replication of "hot" keys onto multiple nodes.

DynamoDB [117] is a fully-managed NoSQL data store from AWS that guarantees consistent performance "at any scale". Based on the popular Dynamo [118] data store, DynamoDB allows customers to declare their desired read/write throughput for a given table, then the underlying infrastructure is configured so to meet these requirements with high probability. DynamoDB is the de-facto industry standard solution for real-time and predictable performance, high reliability and availability, and weak data consistency models.

CitrusLeaf [119] is a NoSQL data store that combines the consistency and reliability capabilities of traditional databases with the extreme scalability of self-sufficient clustered distributed architectures, such as Apache Cassandra and Dynamo. It dynamically redistributes data between replica nodes using real-time prioritization techniques to balance long-running tasks (e.g., batch queries or bookkeeping activities) and short client transactions with sub-millisecond requirements.

The trend of moving the computational resources closer to the "edge" of the network arose to address the concerns of response time requirement, battery life constraint, and bandwidth cost saving. In this context, FogStore [120] is a geo-distributed key-value store based on Apache Cassandra suitable for strongly consistent systems with low latency and fault-tolerance requirements. The work proposes a location-aware replica placement mechanism that tackles the conflicting problem of placing data replicas close to the relevant clients (for low latency), or in distant locations (for fault-tolerance).

An application-specific edge data store has been proposed [121] for distributed machine vision applications. A compute-intensive vision algorithm extracts image feature vectors and key-frames: the former have latency-critical requirements, whereas the latter are kept primarily for archival purposes. The work incorporates an autonomic rate limiter for

[14] https://cassandra.apache.org/

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

transmitting key-frames that sacrifices their latency and accuracy to maintain latency-criticality for feature vectors. Early experimentation with a synthetic workload shows a median latency improvement of 84%.

Portkey [122] is a distributed key-value store considering the time-varying mobility and latency patterns of emerging IoT and mobile applications. It continuously monitors the client locations and data access patterns, and it dynamically devises a near-optimal data placement strategy to control the average/tail latency. With respect to existing NoSQL data stores, Portkey employs self-correcting greedy heuristics that prioritize fast (and frequent) approximate placement decisions over slow optimal placements, reducing tail request latency by 21–82%.

C. Discussion

In the storage research area, there is a lack of works investigating the use of low latency, predictable storage solutions for cloud applications. Investigations on "proper" real-time databases have been of interest only in the restricted domain of hard real-time systems. However, modern cloud-hosted time-critical applications may create an emerging need for novel solutions tied to the cloud computing context. Here, there are several research opportunities thanks to promising new storage technologies, such as glszns SSDs, integration with predictable computing mechanisms as reviewed in Section II, and cloud orchestration solutions as seen in Section III. This leaves room for unexplored research opportunities in the development of cloud-based, data stores for time-critical applications.

V. Networking

Network infrastructures are becoming increasingly suitable for time-critical communications [14], [128], [129] thanks to 5G/6G technologies such as glsurllc, and glstsn. glsurllc can facilitate highly critical applications with very demanding requirements in terms of glse2e latency (millisecond level), reliability, and availability. glstsn aims to provide deterministic services over the IEEE standard 802.3 for Ethernet wired networks. This means guaranteed packet transport with low and bounded latency, low packet delay variation, and low packet loss. Moreover, the recent advent of glsnfv [130], where physical appliances sized for peak hour operations are being replaced by Virtual Network Functions (VNFs), has led network providers to increasingly adopt principles and solutions typical of cloud computing, albeit within a private cloud context. VNFs are software components designed with elasticity and scaling abilities typical of cloud services [131], and deployed on general-purpose data centers of a network operator, typically recurring to cloud platforms.

Time-critical use cases need bounded, low latency, and often deterministic glse2e communications with high reliability and availability. They require a networking infrastructure meeting requirements that are different from normal best-effort, broadband services, namely each packet must meet the timing requirement, otherwise the service will not work properly. In the following, an overview of research works dealing with predictable and reliable networking is made, focusing on: solutions ensuring deterministic traffic scheduling in wired communication networks, along with the challenges for supporting it in wireless networks as well; real-time capabilities of the communication stack, encompassing both OS-level and hypervisor-level predictable packet processing, as needed in virtualized/containerized workloads to meet their timing constraints; and, real-time packet processing capabilities of the virtual switching components, so crucial for the glse2e performance of distributed cloud workloads. The main aspects of the works reviewed below are summarized in Table V.

TABLE V
Characteristics of Different Communication Solutions

Ref.	Solutions	Latency[1]	High through.	Isol.[2]	High avail.[3]
[132]	TSN IEEE 802.1	Predictable	✓	✓	✓
[133], [134]	TSN + DPDK + K8S	Predictable	✓	✓	
[135]	5G TSN/TSC	Predictable	✓	✓	✓
[136]	5G URLLC	Bounded	✓	✓	✓
[137]	Linux - TBS Qdisc	Predictable			
[138]	Linux - PREEMPT-RT	Bounded			
[139]	Virtual Switching	Bounded		✓	
[140]	Virtual Switching	Bounded	✓		
[141]	Virtual Switching	Bounded	✓		

[1]Solutions that provide predictable latency can guarantee a stable, exact latency value. This means also low jitter, hence, solutions with predictable latency can be used for deterministic communication.
[2]In order to exclude spatial interference between applications of different safety-criticalities, the typical solution is to provide applications with dedicated resources to rule out unintended resource interference.
[3]High availability and reliability ($\geq 99.999\%$).

A. Deterministic Networking

Over recent years, a large set of real-time fieldbus protocols were introduced in industrial networks [22], resulting in fragmented segments with proprietary communication solutions. One step towards the convergence in the industrial ecosystem was the usage of the real-time Ethernet variants, such as the old RTnet [63], or more recently EtherCAT [142] and PROFINET [143], that can provide deterministic performance, e.g. guaranteed, bounded latency. glstsn, specified by IEEE 802.1 [132], seems now to be a converged, standardized Ethernet extension, introducing time-sensitive support. Hence, glstsn is expected to replace the different, legacy real-time Ethernet and fieldbus variants. glstsn includes a set of independent standards, which can be applied as a toolset to ensure a deterministic, reliable communication solution for time-critical applications. The IEEE 802.1AS defines the glsgptp, which ensures that the clocks of each end-device can be synchronized with a μs-order precision.

The IEEE 802.1Qbv time-aware scheduling was designed to provide cyclical, pre-defined communication time slices for the different Ethernet glsqos classes. A transmission gate is associated with each queue belonging to a glsqos class, and the queued frames can be selected for transmission only when the state of the gate is open. The configuration plan for the gates (gate control list) is calculated centrally, by the glstsncnc, based on the service requirements and the network characteristics (such as topology, glstsn bridge delay, etc.), then the scheduling plan is pushed to each glstsn bridge. The proper operation of 802.1Qbv

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

requires time synchronization, which ensures that each device can handle the frames belonging to a certain glsqos class in a network-wide, harmonized way. To reduce the end-to-end latency, it is important to move the control/server application instances closer to the client and the edge. The fog computing paradigm/architecture is a good candidate to support this goal.

In [144], glstsn is proposed as the ideal choice for the networking layer of a fog computing-enabled industry automation deployment. The IEEE 802.1Qbv is recommended for handling the time-critical traffic and the related scheduling configuration challenges are overviewed in detail. It is assumed that Industry 4.0 applications require the frequent, dynamic reconfiguration of production processes, and consequently, the re-design of the traffic schedule is often needed. Therefore, a scheduling heuristic is proposed and evaluated, which can be used to reconfigure the transmission schedule at application runtime. A crucial goal in the heuristic is to avoid the interference between the concurrent time-critical streams using the minimum number of queues.

More recently, in [133], an architecture is proposed to integrate deterministic glstsn networking within a Kubernetes-based container management infrastructure, called KuberneTSN. The concept is implemented in a seamless way within a glstsn plug-in compliant with the glscni, which uses also DPDK-based communications as available in glsovs to achieve the lowest possible networking latency on the node. Experimental results with synthetic benchmarks show good potential for achieving lower and more stable networking latency, albeit the gain is in the range of tens of microseconds, in the considered set-up. The KuberneTSN framework is also used in [134] as a basis to build a "vPLC", i.e., a containerized equivalent of a glsplc handed over to a local private cloud, designed with in mind compatibility with open standards for Industry 4.0.

Many Industry 4.0 use cases require wireless connectivity to server mobile devices. In this case, 5G networks promise to be the best option for this purpose. However, the basic 5G glsurllc feature [136] in itself is not enough to ensure the required determinism in communications for serving some time-critical Industrial 4.0 use cases, especially where time-awareness is crucial. Therefore, the gls3gpp started to specify the glstsc service for deterministic/synchronous communications in Release 16, and further enhancements are introduced in Release 17. The gls3gpp glsts 23.501 [135] describes the architecture for providing the seamless integration of 5G networks into the glstsn ecosystem, describing how the 5G System is modeled as a virtual glstsn bridge, the details of the control plane interworking, as well as how the characteristics of time-sensitive communications can be used to optimize the radio scheduling in the 5G domain.

A feasibility study on how a containerized, standard-compliant implementation of the 5G network can support the glsgoose protocol defined by the IEC 61850 standard for time-critical communications in Smart Grid is performed in [145]. Authors show that the average one-way delay between two endpoints is less than 0.6 ms, the average delay of the virtualized core network segment is 0.3 ms, while the maximum is below 0.4 ms (during the measurement period). The conclusion is that 5G network is able to support the time-critical glsgoose traffic. However, the radio transmission - which may have a significant effect on the end-to-end delay - is only emulated in the experiment, so this is a limitation of the work.

In [146], a detailed investigation is presented on how the current framework specified by the gls3gpp for 5G and glstsn integration can support deterministic communications; the paper also presents a quantitative analysis using a closed-loop control application. The paper shows a network scenario where the 5G domain is integrated into a wired glstsn domain as a virtual glstsn bridge (according to the gls3gpp specification), and the packet delay budget and required bitrate for different deterministic, periodic traffic flows are calculated. The work identifies some design principles, such as: only on-premises 5G deployment can fulfill the strict latency requirements; and the adequate glsran resource allocation is crucial, which can be achieved by using (semi-)persistent scheduling on the radio. It is noted that the estimation of the 5G virtual bridge delay is a difficult task, albeit this is one cornerstone of the 5G-glstsn integration.

In [147], a scenario of cooperative work of mobile robots is presented, where tight synchronization is an essential requirement. Therefore, a concept is introduced on how the glstsn time synchronization (IEEE 802.1AS) can be integrated with 5G networks in a seamless way. The essence is to use the synchronization between the gNodeB and the corresponding User Equipments (UEs), since this ensures the required time accuracy, and then calculate the offset between the glstsn and 5G system clocks and adjust the local clocks in the UEs accordingly. It should be mentioned that, even if the proposed concept is validated on a 4G testbed, the measurement results confirm that the achieved accuracy is almost the same as the 802.1AS in a wired network deployment.

B. Real-time Communication Stack

The current trend in cloud application development is moving towards a cloud-native and microservices-based design: instead of deploying a monolithic application, its functionality is split into a multitude of micro-services deployed independently, so that they can exploit to the maximum extent the benefits of being deployed in a cloud environment. Therefore, it becomes paramount to use the appropriate networking abstraction among instances. For example, when using Kubernetes, several implementations of the glscni are available, with a wide variety of performance characteristics [148]. Moreover, in order to ensure the real-time communications for cloudified applications, the real-time support features of the network stack in the virtualized domain (Linux communication stack, container networking) has to be leveraged. Considering the support of time-critical traffic (e.g. glstsn 802.1Qbv scheduled traffic support) in an end-to-end manner, a software solution is required in the end hosts that are compatible with the 802.1Qbv traffic scheduling.

In [149], the implementation of a time-aware shaper using the glstbs Linux Qdisc [137] is presented, showing how to guarantee the schedule of packets to be transmitted. A detailed analysis is performed using a small demo network with 3 glstsn bridges. The results show that, if glstbs Qdisc is applied, the minimum transmission time is slightly increased, but the jitter could be two orders of magnitude lower than without glstbs. The adjustment

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

of the so-called minimum transmit offset, which is the time interval into the future the host application sets the packet transmit time in order to ensure that the packet is sent out in its scheduled time window, is also investigated. The measurements show that, if two hosts are directly interconnected, the transmission time increase is 55% due to the required transmit offset. However, in the case of a 3-bridge network, the increase is only 17.6%. To sum up, the support of time-critical applications requires using glstbs Qdisc, together with a transmission offset, to handle the uncertainties of e.g., the glscpu load due to the software stack. Although this causes some glse2e delay increase, glstsn-grade traffic scheduling is not possible otherwise.

Despite the fact that the Linux communication network subsystem is not optimized for bounded latency, a reasonably deterministic behavior is expected by using the PREEMPT-RT patch [138]. In [150], a comprehensive theoretical and experimental analysis of the Real-time Linux scheduling latency can be found, where the goal is to determine the sources of the latency. In [151], the real-time performance of Linux is reviewed with the usage of PREEMPT-RT in different system configurations. The results show that, although the non-real-time kernel exhibits the best average performance, a huge number of frames exceeds their latency bound, even if there is no background traffic. With PREEMPT-RT, a bounded latency can be achieved even in presence of concurrent traffic, but some missed deadlines can still be detected. Pinning the real-time tasks and the glsirq of the real-time traffic queue to the same glscpu, ensures a bounded latency in a stressed system, with no missed deadlines. If the real-time tasks are performed in an exclusively dedicated glscpu, a similar behavior was detected, but with no further performance improvements. The main conclusion is that PREEMPT-RT is required for real-time applications, but in itself, it is not enough to guarantee bounded latency for each packet, so other system configurations, such as glscpu isolation for real-time processes, are also required.

The authors in [152] emphasize that to execute collaborative tasks using a distributed control system, a large number of individual clocks in the system must be synchronized, and it should also be investigated how the time-based distributed scheduling of the control instances can be performed. The paper discusses in detail the support of the time synchronization in Linux, then measurements are performed using a set of TI Sitara am335x embedded glssoc devices. Authors describe several implementation options for real-time scheduling on Linux for glssoc hardware. The main conclusions are that the most precise solution can be achieved if the system contains glspru, since in this case sub 1 μs scheduling accuracy and precision can be guaranteed. However, it should be noted that this solution requires special programming skills about the Programmable Real-time Units (PRUs), and the code is not portable. Other options, such as HRTIMER service and SCHED-FIFO can provide only 100 μs scheduling accuracy which certainly limits the number of possible applications; the authors favor the simpler SCHED-FIFO based solution for applications with less stringent requirements.

The goal of [153] is to give a solution for the main performance challenge of virtualized environments, i.e., isolating a glsvm running a time-sensitive (e.g., soft real-time) application and a VMs running a bandwidth-intensive application, deployed on the same physical host. The paper shows the details of the transmission/reception as well as the essence of Qdisc capabilities in standard Linux and discusses the details of how the network stack is modified in Xen, considering two representative versions of Linux, e.g., Linux 3.10 and 3.18. From the perspective of prioritizing the time-sensitive traffic, two main limitations of the traffic control in Xen are identified, namely priority inversion between packets originating from the latency-sensitive and bandwidth-intensive domain, and priority inversion between the transmission and reception code paths. The main contribution of the paper is to introduce a glsvatc scheme, which ensures that time-sensitive flows coming from a glsvm are treated as high-priority traffic and assigned to high-priority kernel threads, so the real-time traffic can be protected; the number of priority levels can be configured according to the traffic characteristics. The paper also contains a detailed evaluation of glsvatc using various numbers of bandwidth-intensive background processes, showing that glsvatc can successfully mitigate the traffic control limitations of Xen.

The authors in [154] investigate a practical approach to ensure low latency communications within a distributed, virtualized 5G glsran deployment by introducing UDPDK, a novel open-source middleware on top of the glsdpdk, for easing integration of glsdpdk-based communications. The authors also integrated UDPDK within the glsoai software stack for glsran packet processing. The experimental results show that UDPDK can outperform a standard socket-based glsapi, achieving on average one-tenth of the glsrtt.

Another important aspect for supporting soft real-time services in a virtualized environment is the proper provisioning and management of the virtualized cloud resources considering the real-time constraints. On one hand, authors in [155] formulate the resource provisioning problem using a generic cloud model, a multi-tenant ecosystem, and a service chain deployment. On the other hand, an glsilp-based solution is proposed, called glsnfvrt whose main goal is to maximize the number of service requests by the tenants meeting their bounded latency requirements, while glsvm-level isolation among tenants is ensured. The proposed algorithm performs the initial resource provisioning by determining the required number of service instances for a set of requests, as well as handling the new requests dynamically. The proposed method is evaluated by using simulations (comparing to a greedy heuristic) and using a real testbed. The results show a good performance ofglsnfvrt. However, authors observe that the design of glsnfvrt in reality guarantees that just about 94% of the packets belonging to an accepted service request meet the target deadline. Therefore, it can be used to provision virtualized network resources only for applications with soft real-time requirements.

C. Virtual Switching

Virtual switches are a cornerstone of today's virtualized data centers to enable communications among VMs, containers and also for communications across virtual and physical network

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

segments. As mentioned above, time-sensitive communications can be supported by IEEE glstsn-aware physical switches (considering especially IEEE 802.1Qbv and 802.1Qci). However, realizing a virtual switch that is glstsn-enabled is still an open research challenge.

In [139], a deterministic message-switching solution is proposed by leveraging the glssdn and glstsn paradigms. After the discussion of the requirements of a time-aware virtual switch, the authors propose a detailed system model, where the synchronized operation, bounded relay overhead, and dedicated queues for each message at a specific point in time at the egress ports are emphasized, then the dispatching algorithm is described. A proof-of-concept implementation is also performed, where two virtual switches are deployed and a dedicated (isolated) glscpu core is assigned to each of them; the virtual switch is implemented as a kernel module by leveraging the Linux glsrtai patch. The overall latency caused by the virtual switches is observed in the range of $0.5\ \mu$s to $2\ \mu$s, but several outlier values can also be detected - the major reason behind this was the jitter caused by the glsrtai scheduler.

Due to the strict latency requirements, the glsnfv industry is considering container-based deployments instead of traditional VMs. A detailed comparative study of the different software-based networking solutions for interconnecting glsvnf components in a private cloud infrastructure is presented in [156]. Starting with the overview of the different approaches of inter-container networking (e.g., Kernel-based networking, Kernel bypass, software switches) a detailed experimental analysis is performed using different test setups (intra-host and inter-host container networking). In the analysis, different virtual switch options (glsovs, VPP, VALE, glsdpdk Basic Forwarding Sample Application, SR-IOV) are compared considering latency, throughput, and scalability performance factors. The conclusion is that SR-IOV and VALE outperform the other solutions on a single host not only in the performance parameters but also in terms of the processing power needed to perform high-speed packet forwarding. Considering communication between multiple hosts, the major bottleneck is caused by the glscpu limitation and the throughput limitation of the underlying physical layer.

ESwitch [140] is a novel virtual switch architecture that breaks with general-purpose datapath and embraces a fully customized data plane. ESwitch exploits the fact that most real-world OpenFlow applications compose the same small set of simple, but generic forwarding patterns. So, OpenFlow pipelines can be rewritten in terms of a small set of static templates and then ESwitch uses a template-based code generation to derive the customized datapath. A detailed analysis shows that the ESwitch concept has significant performance gain over OVS, ensuring several times higher raw packet rates, much smaller latency, and predictable throughput even in case of varying workloads.

In [157] and [158], an extensible glsebpf-based datapath architecture for glsovs is proposed. The authors present the glsovs and the glsebpf forwarding models, then discuss the design and implementation of the glsovs datapath considering the restrictions of glsebpf followed by a brief performance analysis. More recently, an experimental comparison [159] in building network functions for packet processing in user-space versus kernel-space has been performed, using the recent `AF_XDP` socket type and glsebpf, as well as recurring to poll-based processing.

One challenge in data-center networks is to provide a proper load-balancing scheme, which ensures that there is no congestion on some links, while others are underutilized. This problem is addressed in [141], which proposes glsvms, a packet load balancing solution that works in the virtual switch layer. The paper discusses in detail the possible load balancing schemes from the viewpoint of deployment location (e.g., centralized, end-host-based), and granularity (e.g., flow-level, packet-level) and concludes that the packet-level granularity should be leveraged to achieve load balancing in the virtual switch layer. The essence of glsvms is to not distribute traffic evenly between available paths (channels) but applies a virtual window size estimation for each channel and to select channels accordingly. VMS can also handle the two main challenges of packet-level load-balancing; packet re-ordering and topology asymmetry-aware traffic distribution. Two implementation alternatives are presented, one based on glsovs, and one based on SmartNIC. The evaluation shows that a near-optimal traffic distribution can be achieved even with an asymmetric topology, with tolerable latency and glscpu utilization overheads in both implementations.

D. Discussion

In the communication area, the glstsn technology and the 3GPP specified 5G-glstsn integration ensure support for time-critical services in both the wired and wireless network domains. Numerous papers overview the real-time features of the Linux communication stack, which appears as a continuously evolving toolset capable of guaranteeing a certain level of timeliness in the cloud networking domain. However, by itself, it is not enough to provide a deterministic glse2e communication: the integrated/harmonized operation of the different networking functions must be ensured through multiple domains, but this area has not been thoroughly explored yet. For instance, harmonization and integration of traffic scheduling and control is needed at the hypervisor/host and guest/container levels. Moreover, handling of compute resources must also be harmonized with traffic scheduling, in order to guarantee that the application deployed in a container is able to generate a certain frame at the proper time, to ensure that it does not miss the target time window.

VI. Conclusion

The industrial and research communities are making steps forward in the various technological areas related to cloud and distributed computing. The acceleration of industrial digitalization and the corresponding emerging use cases are driving the demand for time-critical applications hosted in cloud environments. This paper surveyed the current state of the art concerning time-criticality in the cloud and the features to support time-critical applications. We provided a summary of key challenges in designing cloud infrastructures for time-critical services, covering aspects related to compute, orchestration, storage, and communications. Concerning the two questions posed at the

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

beginning of the paper about the state of time-criticality in cloud computing, we can conclude that:

1) The main challenge for time-criticality in cloud technologies is ensuring temporal protection in presence of multi-tenancy. Most cloud services and infrastructures are still designed only to provide best-effort glsqos. The focus is on maximizing overall throughput, neglecting the issue of providing predictable response times to the individual requests. However, incorporating time predictability properties into the cloud stack is essential to meet the requirements of many emerging time-critical applications (see Table I).

2) There are several solutions for high reliability and time-predictability within each domain. For example, recent developments in networking technologies, are becoming increasingly deterministic, with solutions like glsurllc and glstsn. However, there is a general lack of cross-domain efforts: for instance, despite the existence of hypervisors providing predictable CPU scheduling, no orchestration tools utilize such parameters to properly orchestrate the placement and resource provisioning of time-critical workloads, as briefly discussed in Section II-D. Another example is the one of real-time processing platforms: most of them focus on efficient process placement and resource allocation without taking into account storage-related matters, such as controlling the I/O backlog.

Regarding possible future directions, we highlighted that most of the academic papers found in the literature focus on low-latency, and only a few provide methods to guarantee a certain latency with high probability in distributed systems. Moreover, very few papers (mostly in the compute area) are backed up by real-time theory. In a real-world, time-critical scenario, a theoretical foundation is essential to assess the practicality of a time-critical solution. Hence the need for further investigations towards more formal evaluations of timing properties and guarantees. A promising research direction lies in the storage domain, which remains the most neglected area in modern literature, despite the existence of DynamoDB, the only cloud service with reasonably predictable performance guarantees.

In conclusion, no work in the literature proposes a comprehensive cloud system that incorporates time-critical guarantees within the entire cloud stack. Similarly, no commercial clouds integrate real-time configurations and, therefore, there are no ready-to-use time-critical cloud solutions yet.

REFERENCES

[1] G. Simson and A. Harold, *Architects of the Information Society: Thirty-Five Years of the Laboratory for Computer Science at MIT.* Cambridge, MA, USA: MIT Press, 1999.

[2] M. Armbrust et al., "Above the clouds: A Berkeley view of cloud computing," Univ. of California, Tech. Rep. UCB/EECS-2009-28, 2009.

[3] M. Armbrust et al., "A view of cloud computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[4] E. Jonas et al., "Cloud programming simplified: A Berkeley view on serverless computing," Univ. California, Berkeley, Tech, Rep. UCB/EECS-2019-3, 2019. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.html

[5] I. Baldini et al., *Serverless Computing: Current Trends and Open Problems*. Singapore: Springer, 2017, pp. 1–20. [Online]. Available: https://doi.org/10.1007/978-981-10-5026-8_1

[6] C. Cicconetti, M. Conti, A. Passarella, and D. Sabella, "Toward distributed computing environments with serverless solutions in edge systems," *IEEE Commun. Mag.*, vol. 58, no. 3, pp. 40–46, Mar. 2020.

[7] J. Schleier-Smith et al., "What serverless computing is and should become: The next phase of cloud computing," *Commun. ACM*, vol. 64, no. 5, pp. 76–84, 2021. [Online]. Available: https://doi.org/10.1145/3406011

[8] M. Gundall et al., "Introduction of a 5G-enabled architecture for the realization of industry 4.0 use cases," *IEEE Access*, vol. 9, pp. 25508–25521, 2021.

[9] F. Alriksson, L. Boström, J. Sachs, Y.-P. E. Wang, and A. Zaidi, "Critical IoT connectivity ideal for time-critical communications," *Ericsson Technol. Rev.*, vol. 2020, no. 6, pp. 2–13, 2020.

[10] F. Alriksson, D. H. Kang, C. Phillips, J. L. Pradas, and A. Zaidi, "XR and 5G: Extended reality at scale with time-critical communication," *Ericsson Technol. Rev.*, vol. 2021, no. 8, pp. 2–13, 2021.

[11] "Time-critical communication: Leading the next wave of 5G innovation," Ericsson, Tech. Rep., 2021.

[12] A. Balador et al., "Aorta: Advanced offloading for real-time applications," in *Proc. Real-Time Clouds (RT-Clouds) Workshop, Co-Located 2023 Euromicro Conf. Real-Time Syst.*, Vienna, Austria, Jul. 2023. [Online]. Available: https://lup.lub.lu.se/record/8fc5c69a-25e1-4ce8-9b4d-2211923e36a0

[13] 5G-ACIA White Paper, "5G for connected industries and automation (second edition)," 5G Alliance for Connected Industries and Automation, ZVEI – German Electrical and Electronic Manufacturers' Association 5G Alliance for Connected Industries and Automation (5G-ACIA), Feb. 2019. [Online]. Available: https://5g-acia.org/13b01bc3-8aa9-40b8-998f-1975ea111402

[14] M. Uusitalo et al., "European vision for the 6G network ecosystem," Nov. 2024, doi: 10.5281/zenodo.14230482.

[15] S. A. Ashraf et al., "5G-powered FRMCS," Ericsson White Paper GFTL-22:000375, Mar. 2022. [Online]. Available: https://na.experiences.ericsson.net/5g-powered-frmcs

[16] J. Danielsson, N. Tsog, and A. Kunnappilly, "A systematic mapping study on real-time cloud services," in *Proc. IEEE/ACM Int. Conf. Utility Cloud Comput. Companion*, 2018, pp. 245–251.

[17] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[18] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[19] M. García-Valls, T. Cucinotta, and C. Lu, "Challenges in real-time virtualization and predictable cloud computing," *J. Syst. Archit.*, vol. 60, no. 9, pp. 726–740, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762114001015

[20] D. Kyriazis, T. A. Varvarigou, and K. Konstanteli, "Achieving real-time in distributed computing - from grids to clouds," in *Achieving Real-Time in Distributed Computing*. Hershey, PA, USA: IGI Global, 2011.

[21] M. Hasan and M. S. Goraya, "Fault tolerance in cloud computing environment: A systematic survey," *Comput Ind.*, vol. 99, pp. 156–172, 2018.

[22] P. Danielis, H. Puttnies, E. Schweissguth, and D. Timmermann, "Real-time capable internet technologies for wired communication in the industrial IoT-a survey," in *Proc. IEEE 23rd Int. Conf. Emerg. Technol. Factory Automat.*, 2018, pp. 266–273.

[23] V. Struhár, M. Behnam, M. Ashjaei, and A. V. Papadopoulos, "Real-time containers: A survey," in *Proc. 2nd Workshop Fog Comput IoT*, 2020, pp. 7:1–7:9. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2020/12001

[24] M. Afrin, J. Jin, A. Rahman, A. Rahman, J. Wan, and E. Hossain, "Resource allocation and service provisioning in multi-agent cloud robotics: A comprehensive survey," *IEEE Commun. Surv. Tuts.*, vol. 23, no. 2, pp. 842–870, Second Quarter 2021.

[25] R. Queiroz, T. Cruz, J. Mendes, P. Sousa, and P. Simões, "Container-based virtualization for real-time industrial systems–a systematic review," *ACM Comput. Surv.*, vol. 56, no. 3, pp. 1–38, 2023, doi: 10.1145/3617591.

[26] L. Abeni, A. Goel, C. Krasic, J. Snow, and J. Walpole, "A measurement-based analysis of the real-time performance of linux," in *Proc. 8th IEEE Real-Time Embedded Technol. Appl. Symp.*, 2002, pp. 133–142.

[27] M. Panahi, W. Nie, and K.-J. Lin, "A framework for real-time service-oriented architecture," in *Proc. IEEE Conf. Commerce Enterprise Comput.*, 2009, pp. 460–467.

[28] T. Cucinotta et al., "A real-time service-oriented architecture for industrial automation," *IEEE Trans. Ind. Informat.*, vol. 5, no. 3, pp. 267–277, Aug. 2009.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

[29] J. Liedtke, H. Hartig, and M. Hohmuth, "Os-controlled cache predictability for real-time systems," in *Proc. 3rd IEEE Real-Time Technol. Appl. Symp.*, 1997, pp. 213–224.
[30] M. Lee, A. S. Krishnakumar, P. Krishnan, N. Singh, and S. Yajnik, "Supporting soft real-time tasks in the XEN hypervisor," in*Proc. 6th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, 2010, Art. no. 97. [Online]. Available: http://portal.acm.org/citation.cfm?doid=1735997.1736012
[31] S. Xi, J. Wilson, C. Lu, and C. Gill, "RT-Xen: Towards real-time hypervisor scheduling in XEN," in *Proc. 9th ACM Int. Conf. Embedded Softw.*, 2011, pp. 39–48.
[32] F. Hofer, M. A. Sehr, A. Iannopollo, I. Ugalde, A. Sangiovanni-Vincentelli, and B. Russo, "Industrial control via application containers: Migrating from bare-metal to IAAS," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci.*, 2019, pp. 62–69.
[33] T. Cucinotta et al., "Virtualised e-learning with real-time guarantees on the IRMOS platform," in *Proc. IEEE Int. Conf. Serv.-Oriented Comp. Appl.*, 2010, pp. 1–8.
[34] L. Abeni, A. Biondi, and E. Bini, "Hierarchical scheduling of real-time tasks over linux-based virtual machines," *J. Syst. Softw.*, vol. 149, pp. 234–249, 2019.
[35] L. Abeni and D. Faggioli, "An experimental analysis of the XEN and KVM latencies," in *Proc. IEEE 22nd Int. Symp. Real-Time Distrib. Comput.*, 2019, pp. 18–26. [Online]. Available: https://ieeexplore.ieee.org/document/8759339/
[36] L. Abeni and D. Faggioli, "Using XEN and KVM as real-time hypervisors," *J. Syst. Archit.*, vol. 106, 2020, Art. no. 101709. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762120300035
[37] A. Agache et al., "Firecracker: Lightweight virtualization for serverless applications," in *Proc. 17th USENIX Symp. Netw. Syst. Des. Implementation*, 2020, pp. 419–434.
[38] Y. Li, J. Zhang, C. Jiang, J. Wan, and Z. Ren, "PINE: Optimizing perf. isolation in container env," *IEEE Access*, vol. 7, pp. 30410–30422,2019. [Online]. Available: https://ieeexplore.ieee.org/document/8648379/
[39] J. Wu and T.-I. Yang, "Dynamic CPU allocation for docker containerized mixed-criticality real-time systems," in *Proc. IEEE Int. Conf. Appl. Syst. Invention*, 2018, pp. 279–282. [Online]. Available: https://ieeexplore.ieee.org/document/8394587/
[40] L. Abeni, A. Balsini, and T. Cucinotta, "Container-based real-time scheduling in the linux kernel," *ACM SIGBED Rev.*, vol. 16, no. 3, pp. 33–38, 2019. [Online]. Available: http://retis.santannapisa.it/~tommaso/publications/EWILI-2018-RT-LXC.pdf
[41] T. Cucinotta, L. Abeni, M. Marinoni, R. Mancini, and C. Vitucci, "Strong temporal isolation among containers in OpenStack for NFV services," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 763–778, First Quarter 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9551719/
[42] C.-N. Mao et al., "Minimizing latency of real-time container cloud for software radio access networks," in *Proc. 7th Intern. Conf. Cloud Comput. Technol. Sci.*, 2015, pp. 611–616. [Online]. Available: http://ieeexplore.ieee.org/document/7396222/
[43] M. Cinque and D. Cotroneo, "Towards lightweight temporal and fault isolation in mixed-criticality systems with real-time containers," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. Workshops*, 2018, pp. 59–60. [Online]. Available: https://ieeexplore.ieee.org/document/8416214/
[44] M. Cinque, R. D. Corte, A. Eliso, and A. Pecchia, "RT-CASEs: Container-based virtualization for temporally separated mixed-criticality task sets," in *Proc. 31st Euromicro Conf. Real-Time Syst.*, 2019, pp. 5:1–5:22. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECRTS.2019.5
[45] M. Cinque, R. D. Corte, and R. Ruggiero, "Preventing timing failures in mixed-criticality clouds with dynamic real-time containers," in *Proc. IEEE 17th Eur. Dependable Comput. Conf.*, 2021, pp. 17–24. [Online]. Available: https://ieeexplore.ieee.org/document/9603441/
[46] T. Tasci, J. Melcher, and A. Verl, "A container-based architecture for real-time control applications," in *Proc. IEEE Int. Conf. Eng. Technol. Innov.*, 2018, pp. 1–9. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8436369
[47] M. Barletta, M. Cinque, L. De Simone, and R. D. Corte, "Achieving isolation in mixed-criticality industrial edge systems with real-time containers," in *Proc. 34th Euromicro Conf. Real-Time Syst.*, 2022, pp. 15:1–15:23. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECRTS.2022.15
[48] T. Kurek, "Unikernel network functions: A journey beyond the containers," *IEEE Commun. Mag.*, vol. 57, no. 12, pp. 15–19, Dec. 2019. [Online]. Available:https://ieeexplore.ieee.org/document/8930819/
[49] C. Mistry, B. Stelea, V. Kumar, and T. Pasquier, "Demonstrating the practicality of unikernels to build a serverless platform at the edge," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci.*, 2020, pp. 25–32. [Online]. Available: https://ieeexplore.ieee.org/document/9407311/
[50] K.-H. Chen, M. Günzel, B. Jablkowski, M. Buschhoff, and J.-J. Chen, "Unikernel-based real-time virtualization under deferrable servers: Analysis and realization," in *Proc. 34th Euromicro Conf. Real-Time Syst.*, 2022, pp. 6:1–6:22. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2022/16323
[51] I. Shin and I. Lee, "Compositional real-time scheduling framework," in *Proc. 25th IEEE Int. Real-Time Syst. Symp.*, 2004, pp. 57–67.
[52] I. Shin and I. Lee, "Compositional real-time scheduling framework with periodic model," *ACM Trans. Embed. Comput. Syst.*, vol. 7, no. 3, May 2008, Art. no. 30. [Online]. Available:https://doi.org/10.1145/1347375.1347383
[53] T. Cucinotta, L. Abeni, M. Marinoni, and C. Vitucci, "The importance of being OS-aware–in performance aspects of cloud computing research," in *Proc. 8 th Int. Conf. Cloud Comput. Serv. Sci.*, SciTePress, 2018, pp. 626–633, doi: 10.5220/0006793906260633.
[54] L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," in *Proc. IEEE Real-Time Syst. Symp.*, 1998, pp. 4–13.
[55] H. Li, X. Xu, J. Ren, and Y. Dong, "ACRN: A big little hypervisor for IoT development," in *Proc. ACM 15th Int. Conf. Virtual Execution Environments*, 2019, pp. 31–44. [Online]. Available: http://doi.acm.org/10.1145/3313808.3313816
[56] R. Ramsauer, J. Kiszka, D. Lohmann, and W. Mauerer, "Look mum, no VM exits!(Almost)," in *Proc. 13th Workshop Operating Syst. Platforms Embedded Real-Time Appl.*, 2017, pp. 13–18.
[57] R. West, Y. Li, E. Missimer, and M. Danish, "A virtualized separation kernel for mixed-criticality systems," *ACM Trans. Comput. Syst.*, vol. 34, no. 3, pp. 1–41, Jun. 2016.
[58] J. Martins, A. Tavares, M. Solieri, M. Bertogna, and S. Pinto, "Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems," in *Proc. Workshop Next Gener. Real-Time Embedded Syst.*, , Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2020, pp. 3:1–3:14.
[59] A. Crespo, I. Ripoll, and M. Masmano, "Partitioned embedded architecture based on hypervisor: The xtratum approach," in *Proc. Eur. Dependable Comput. Conf.*, 2010, pp. 67–72.
[60] G. Klein et al., "seL4: Formal verification of an OS kernel," in *Proc. ACM SIGOPS 22nd Symp. Operating Syst. Princ.*, 2009, pp. 207–220. [Online]. Available: https://doi.org/10.1145/1629575.1629596
[61] L. Orciari, D. Raggini, and A. Tilli, "Taming edge computing for hard real-time advanced control of mechatronic systems," *IEEE Trans. Ind. Informat.*, vol. 20, no. 8, pp. 9898–9906, Aug. 2024.
[62] A. F. Ocampo, M.-R. Fida, J. F. Botero, A. Elmokashfi, and H. Bryhni, "Opportunistic CPU sharing in mobile edge computing deploying the cloud-ran," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2201–2217, Oct. 2023.
[63] J. Kiszka and B. Wagner, "RTNet - a flexible hard real-time networking framework," in *Proc. IEEE Conf. Emerg. Technol. Factory Automat.*, 2005, pp. 8 pp.–456.
[64] S. Chen and M. Zhou, "Evolving container to unikernel for edge computing and applications in process industry," *Processes*, vol. 9, no. 2, Feb. 2021 Art. no. 351. [Online]. Available:https://www.mdpi.com/2227-9717/9/2/351
[65] I. Mavridis and H. Karatza, "Lightweight virtualization approaches for software-defined systems and cloud computing: An evaluation of unikernels and containers," in *Proc. IEEE 6th Int. Conf. Softw. Defined Syst.*, 2019, pp. 171–178. [Online]. Available: https://ieeexplore.ieee.org/document/8768586/
[66] R. Behravesh, E. Coronado, and R. Riggio, "Performance evaluation on virtualization technologies for NFV deployment in 5G networks," in *Proc. IEEE Conf. Netw. Softwarization*, 2019, pp. 24–29. [Online]. Available: https://ieeexplore.ieee.org/document/8806664/
[67] I. Mavridis and H. Karatza, "Orchestrated sandboxed containers, unikernels, and virtual machines for isolation-enhanced multitenant workloads and serverless computing in cloud," *Concurrency Comput. Pract Exper*, vol. 35, 2021, Art. no. e6365. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1002/cpe.6365
[68] A. Tzenetopoulos, D. Masouros, S. Xydis, and D. Soudris, "Interference-aware orchestration in kubernetes," in *Proc. Int. Conf. High Perform. Comput.*, Springer, 2020, pp. 321–330.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

[69] L. Toka, "Ultra-reliable and low-latency computing in the edge with kubernetes," *J. Grid Comput.*, vol. 19, no. 3, pp. 1–23, 2021. [Online]. Available: https://link.springer.com/article/10.1007/s10723-021-09573-z

[70] S. Xi et al., "RT-Open stack: CPU resource management for real-time cloud computing," in *Proc. IEEE 8th Int. Conf. Cloud Comput.*, 2015, pp. 179–186. [Online]. Available: http://ieeexplore.ieee.org/document/7214043/

[71] S. Fiori, L. Abeni, and T. Cucinotta, "RT-Kubernetes: Containerized real-time cloud computing," in *Proc. 37th ACM/SIGAPP Symp. Appl. Comput.*, 2022, pp. 36–39. [Online]. Available: https://dl.acm.org/doi/10.1145/3477314.3507216

[72] V. Struhár, S. S. Craciunas, M. Ashjaei, M. Behnam, and A. V. Papadopoulos, "REACT: Enabling real-time container orchestration," in *Proc. 26th IEEE Int. Conf. Emerg. Technol. Factory Automat.*, 2021, pp. 1–8.

[73] V. Struhár, S. S. Craciunas, M. Ashjaei, M. Behnam, and A. V. Papadopoulos, "Hierarchical resource orchestration framework for real-time containers," *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 1, pp. 1–24, Jan. 2024. [Online]. Available:https://doi.org/10.1145/3592856

[74] F. Lumpp, F. Fummi, H. D. Patel, and N. Bombieri, "Enabling kubernetes orchestration of mixed-criticality softw. for autonomous mobile robots," *IEEE Trans. Robot.*, vol. 40, pp. 540–553, 2024.

[75] M. Barletta, M. Cinque, L. De Simone, and R. D. Corte, "Criticality-aware monitoring and orchestration for containerized industry 4.0 environments," *ACM Trans. Embedded Comp. Sys.*, vol. 23, no. 1, pp. 1–28, 2024.

[76] H. Fan et al., "Accelerating parallel applications in cloud platforms via adaptive time-slice control," *IEEE Trans. Comput.*, vol. 70, no. 7, pp. 992–1005, 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9107394

[77] H. Chen, X. Zhu, H. Guo, J. Zhu, X. Qin, and J. Wu, "Towards energy-efficient scheduling for real-time tasks under uncertain cloud computing environment," *J. Sys. Softw.*, vol. 99, pp. 20–35, 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121214001903

[78] H. Li, C. Lu, and C. Gill, "Predicting latency distributions of aperiodic time-critical services," in *Proc. IEEE Real-Time Syst. Symp.*, 2019, pp. 30–42. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9052192

[79] M. Kogias and E. Bugnion, "HovercRaft: Achieving scalability and fault-tolerance for microsecond-scale datacenter services," in *Proc. 15th Eur. Conf. Comput. Syst.*, 2020, pp. 1–17.

[80] J. Geng, A. Sivaraman, B. Prabhakar, and M. Rosenblum, "Nezha: Deployable and high-performance consensus using synchronized clocks," *Proc. VLDB Endowment*, vol. 16, no. 4, pp. 629–642, Dec. 2022. [Online]. Available:https://doi.org/10.14778/3574245.3574250

[81] M. K. Aguilera, N. Ben-David, R. Guerraoui, V. J. Marathe, A. Xygkis, and I. Zablotchi, "Microsecond consensus for microsecond applications," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation*, 2020, pp. 599–616. [Online]. Available:https://www.usenix.org/conference/osdi20/presentation/aguilera

[82] H. D. Nguyen, C. Zhang, Z. Xiao, and A. A. Chien, "Real-time serverless: Enabling application performance guarantees," in*Proc. ACM 5th Int. Workshop Serverless Comput.*, 2019, pp. 1–6, doi: 10.1145/3366623.3368133.

[83] M. Szalay, P. Mátray, and L. Toka, "Real-time task scheduling in a FaaS cloud," in *Proc. IEEE 14th Intl. Conf. Cloud Comput.*, 2021, pp. 497–507. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9582186

[84] M. Szalay, P. Matray, and L. Toka, "Real-time FaaS: Towards a latency bounded serverless cloud," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1636–1650, Second Quarter 2023.

[85] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi, "Naiad: A timely dataflow system," in *Proc. 24th ACM Symp. Operating Syst. Princ.*, 2013, pp. 439–455. [Online]. Available: https://doi.org/10.1145/2517349.2522738

[86] X. Fu, T. Ghaffar, J. C. Davis, and D. Lee, "EdgeWise: A better stream processing engine for the edge," in *Proc. USENIX Annu. Tech. Conf.*, 2019, pp. 929–946. [Online]. Available: http://www.usenix.org/conference/atc19/presentation/fu

[87] J. Xu, B. Palanisamy, Q. Wang, H. Ludwig, and S. Gopisetty, "Amnis: Optimized stream processing for edge computing," *J. Parallel Distrib. Comput.*, vol. 160, pp. 49–64, 2022.

[88] H. D. Nguyen and A. A. Chien, "Storm-RTS: Stream processing with stable perf. for multi-cloud and cloud-edge," in *Proc. IEEE 16th Int. Conf. Cloud Comp.*, 2023, pp. 45–57.

[89] S. K. Mishra, B. Sahoo, and P. P. Parida, "Load balancing in cloud computing: A. big picture," *J. King Saud Univ. - Comput. Info. Sci.*, vol. 32, no. 2, pp. 149–158, Feb. 2020. [Online]. Available:https://linkinghub.elsevier.com/retrieve/pii/S1319157817303361

[90] M. Catillo, U. Villano, and M. Rak, "A survey on auto-scaling: How to exploit cloud elasticity," *Int. J. Grid Util. Comput.*, vol. 14, no. 1, pp. 37–50, Jan. 2023. [Online]. Available:https://doi.org/10.1504/ijguc.2023.129702

[91] T. Cucinotta et al., "The IRMOS/ISONI real-time cloud infrastructure: A virtualised e-learning case-study," *IEEE Multimedia Commun. Techn. Committee*, vol. 6, no. 12, 2011, pp. 29–32.

[92] K. Konstanteli, T. Cucinotta, K. Psychas, and T. A. Varvarigou, "Elastic admission control for federated cloud services," *IEEE Trans. Cloud Comput.*, vol. 2, no. 3, pp. 348–361, Third Quarter 2014.

[93] C. Kathpal and R. Garg, "Survey on Fault-Tolerance-Aware Scheduling in Cloud Computing," in *Proc. Info. Commun. Technol. Competitive Strategies*, Springer, 2019, pp. 275–283.

[94] M. Barletta, M. Cinque, C. Di Martino Z. T. Kalbarczyk, and R. K. Iyer, "Mutiny! how does kubernetes fail, and what can we do about it?," in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2024, pp. 1–14. [Online]. Available:https://doi.ieeecomputersociety.org/10.1109/DSN58291.2024.00016

[95] M. Barletta, L. De Simone, R. D. Corte, and C. Di Martino, "Failover timing analysis in orchestrating container-based critical applications," in *Proc. 19th Eur. Dependable Comput. Conf.*, 2024, pp. 81–84. [Online]. Available:https://doi.ieeecomputersociety.org/10.1109/EDCC61798.2024.00026

[96] P. Skarin, W. Tärneberg, K.-E. Årzén, and M. Kihl, "Control-over-the-cloud: A. performance study for cloud-native, critical control systems," in *Proc. IEEE/ACM 13th Int. Conf. Utility Cloud Comput.*, 2020, pp. 57–66.

[97] X. Xu, "From Cloud Comp. to Cloud Manuf," *Robot. Comput.-Integr. Manuf.*, vol. 28, no. 1, pp. 75–86, 2012. [Online]. Available: https://ericsson.sharepoint.com/:b:/r/sites/NewCapabilitiesandEcosystemsProgram/SharedDocuments/AREAS/Technicalareas/TA2-IntelligentNetworkandServiceOperations/TA2Activities/047-TimeCriticalCloudSurvey/Papers-Per/Xu2012.pdf?csf=1&web=1&e=gAffLN

[98] M. Primorac, "Understanding and mitigating latency variability of latency-critical applications," 2020. [Online]. Available: http://infoscience.epfl.ch/record/278200

[99] L. Abeni, R. Andreoli, H. Gustafsson, M. Raquel, and T. Cucinotta, "Fault tolerance in real-time cloud computing," in *Proc. IEEE 26th Int. Symp. Real-Time Distrib. Comput.*, 2023, pp. 170–175.

[100] R. Andreoli, H. Gustafsson, L. Abeni, R. Mini, and T. Cucinotta, "Optimal deployment of cloud-native applications with fault-tolerance and time-critical end-to-end constraints," in *Proc. 16th IEEE/ACM Int. Conf. Cloud Comput.*, 2023, pp. 1–10.

[101] R. Andreoli, H. Gustafsson, L. Abeni, R. Mini, and T. Cucinotta, "Design-time analysis of time-critical and fault-tolerance constraints in cloud services," in *Proc. IEEE 16th Int. Conf. Cloud Comput.*, 2023, pp. 415–417.

[102] P. Valente and F. Checconi, "High throughput disk scheduling with fair bandwidth distribution," *IEEE Trans. Comput.*, vol. 59, no. 9, pp. 1172–1186, Sep. 2010.

[103] M. Bjørling, J. Gonzalez, and P. Bonnet, "LightNVM: The linux open-channel SSD subsystem," in *Proc. 15th USENIX Conf. File Storage Technol.*, 2017, pp. 359–374.

[104] M. Bjørling et al., "ZNS: Avoiding the block interface tax for flash-based SSDs," in *Proc. USENIX Annu. Tech. Conf.*, 2021, pp. 689–703.

[105] H. Litz, J. Gonzalez, A. Klimovic, and C. Kozyrakis, "RAIL: Predictable, low tail latency for NVMe flash," *ACM Trans. Storage*, vol. 18, no. 1, pp. 1–21, 2022. [Online]. Available: https://anakli.inf.ethz.ch/papers/rail_tos.pdf

[106] G. Lee, S. Shin, and J. Jeong, "Efficient hybrid polling for ultra-low latency storage devices," *J. Syst. Archit.*, vol. 122, 2022, Art. no. 102338.

[107] S. Wu, Z. Huang, P. Chen, H. Fan, S. Ibrahim, and H. Jin, "Container-aware I/O stack: Bridging the gap between container storage drivers and solid state devices," in *Proc. 18th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environments*, 2022, pp. 18–30.

[108] Y. Sun, J. Lei, S. Shin, and H. Lu, "Baoverlay: A block-accessible overlay file system for fast and efficient container storage," in *Proc. 11th ACM Symp. Cloud Comput.*, 2020, pp. 90–104.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

[109] H. Fan, S. Bian, S. Wu, S. Jiang, S. Ibrahim, and H. Jin, "Gear: Enable efficient container storage and deployment with a new image format," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 115–125.

[110] K.-D. Kang, J. Oh, and S. H. Son, "Chronos: Feedback control of a real database sys. perf," in *Proc. 28th IEEE Int. Real-Time Syst. Symp.*, 2007, pp. 267–276. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/4408311

[111] W. Kang, S. H. Son, and J. A. Stankovic, "QeDB: A quality-aware embedded real-time database," in *Proc. 15th IEEE Real-Time Embedded Technol. Appl. Symp.*, 2009, pp. 108–117.

[112] C. Xu, F. Xia, M. A. Sharaf, M. Zhou, and A. Zhou, "AQUAS: A quality-aware scheduler for NoSQL data stores," in *Proc. IEEE 30th Int. Conf. Data Eng.*, 2014, pp. 1210–1213. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6816743

[113] C. Xu, M. A. Sharaf, X. Zhou, and A. Zhou, "Quality-aware schedulers for weak consistency key-value data stores," *Distribut. Parallel Databases*, vol. 32, no. 4, pp. 535–581, 2014. [Online]. Available: https://link.springer.com/article/10.1007/s10619-013-7136-4

[114] G. Németh, D. Géhberger, and P. Mátray, "DAL: A locality-optimizing distrib. shared memory sys," in *Proc. 9th USENIX Workshop Hot Topics Cloud Comput.*, 2017, Art. no. 12.

[115] R. Andreoli, T. Cucinotta, and D. B. DeOliveira, "Priority-driven differentiated performance for NoSQL database-as-a-service," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3469–3482, Fourth Quarter 2023.

[116] C. Wu, V. Sreekanti, and J. M. Hellerstein, "Autoscaling tiered cloud storage in anna," *VLDB J.*, vol. 30, no. 1, pp. 25–43, 2021.

[117] S. Perianayagam et al., "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annu. Techn. Conf.*, 2022, pp. 1037–1048. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/vig

[118] G. DeCandia et al., "Dynamo: Amazon's highly available key-value store," *SIGOPS Oper. Syst. Rev.*, vol. 41, no. 6, pp. 205–220, Oct. 2007. [Online]. Available:https://doi.org/10.1145/1323293.1294281

[119] V. Srinivasan and B. Bulkowski, "Citrusleaf: A real-time NoSQL DB which preserves ACID," *VLDB Endowment*, vol. 4, no. 12, pp. 1340–1350, 2011.

[120] H. Gupta and U. Ramachandran, "FogStore: A geo-distributed key-value store guaranteeing low latency for strongly consistent access," in *Proc. 12th ACM Int. Conf. Distrib. Event-Based Syst.*, 2018, pp. 148–159. [Online]. Available: https://www.cc.gatech.edu/grads/h/hgupta40/publication_resources/papers/fogstore.pdf

[121] A. Ravindran and A. George, "An edge datastore architecture for latency-critical distributed machine vision applications," in *Proc. USENIX Workshop Hot Topics Edge Comput.*, 2018, pp. 1–6.

[122] J. Noor, M. Srivastava, and R. Netravali, "Portkey: Adaptive key-value placement over dynamic edge networks," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 197–213, doi: 10.1145/3472883.3487004.

[123] B. Kao and H. Garcia-Molina, "An overview of real-time database systems," in *Proc. Real Time Comput.*, 1994, pp. 261–282. [Online]. Available: http://ilpubs.stanford.edu:8090/39/1/1993-6.pdf

[124] P. Yu, K.-L. Wu, K.-J. Lin, and S. Son, "On real-time databases: Concurrency control and scheduling," *Proc. IEEE*, vol. 82, no. 1, pp. 140–157, Jan. 1994.

[125] J. Lindström, "Optimistic concurrency control methods for real-time database systems," 2003. [Online]. Available: https://helda.helsinki.fi/bitstream/handle/10138/21377/optimist.pdf?sequence=1

[126] A. Magalhaes, J. M. Monteiro, and A. Brayner, "Main memory database recovery: A survey," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–36, Mar. 2021, doi: 10.1145/3442197.

[127] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM Sigmod Rec.*, vol. 39, no. 4, pp. 12–27, 2011. [Online]. Available: https://cgi.di.uoa.gr/~ad/M149/Cattell.pdf

[128] G. A. W. Paper, "Integration of 5G with time-sensitive networking for industrial communications," 5G-ACIA White Paper. 5G Alliance for Connected Industries and Automation. ZVEI German Electrical and Electronic Manufacturers Association 5G Alliance for Connected Industries and Automation (5G-ACIA), 2021. [Online]. Available: https://5g-acia.org/whitepapers/integration-of-5g-with-time-sensitive-networking-for-industrial-communications/

[129] "First report on 5G network architecture options and assessments," 5G SMART, Tech. Rep., 2020.

[130] ETSI, "Network functions virtualisation," in *Proc. White Paper 1. SDN Openflow World Congr.*, 2012. [Online]. Available: https://portal.etsi.org/NFV/NFV_White_Paper.pdf

[131] R. Szabó, M. Kind, F.-J. Westphal, H. Woesner, D. Jocha, and A. Császar, "Elastic network functions: Opportunities and challenges," *IEEE Netw.*, vol. 29, no. 3, pp. 15–21, May/Jun. 2015.

[132] "Time-sensitive networking (TSN) task group," [Online]. Available: https://1.ieee802.org/tsn/

[133] A. Garbugli, L. Rosa, A. Bujari, and L. Foschini, "KuberneTSN: A deterministic overlay network for time-sensitive containerized environments," in *Proc. IEEE Int. Conf. Commun.*, 2023, pp. 1494–1499.

[134] L. Rosa, A. Garbugli, L. Patera, and L. Foschini, "Supporting VPLC networking over TSN with kubernetes in industry 4.," in *Proc. ACM 1st Workshop Enhanced Netw. Techn. Technol. Ind. IoT Cloud Continuum*, 2023, pp. 15–21, doi: 10.1145/3609389.3610566.

[135] "The 3GPP technical specification (TS) 23.501 - time sensitive communication (TSC)," Oct. 2020. [Online]. Available: https://www.3gpp.org/DynaReport/23501.htm

[136] "Study on enhancement of ultra-reliable low-latency communication (URLLC) support in the 5G core network (5GC)," Jun. 2019. [Online]. Available: https://www.3gpp.org/DynaReport/23725.htm

[137] "Time-based scheduling (TBS) Linux qdisc," 2019. [Online]. Available: https://tsn.readthedocs.io/qdiscs.html

[138] "The linux real time collaborative project - PREEMPT_RT patches," [Online]. Available: https://wiki.linuxfoundation.org/realtime/start

[139] H. Fang and R. Obermaisser, "Virtual switch for integrated real-time systems based on SDN," in *Proc. IEEE 6th Int. Conf. Internet Things Syst. Manage. Secur.*, 2019, pp. 344–351. [Online]. Available: https://ieeexplore.ieee.org/document/8939207/

[140] L. Molnár et al., "Dataplane specialization for high-performance OpenFlow software switching," in *Proc. ACM SIGCOMM Conf.*, 2016, pp. 539–552. [Online]. Available:https://dl.acm.org/doi/10.1145/2934872.2934887

[141] Y. Zhang, J. Bi, Z. Li, Y. Zhou, and Y. Wang, "VMS: Load balancing based on the virtual switch layer in datacenter networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1176–1190, Jun. 2020. [Online]. Available:https://ieeexplore.ieee.org/document/9060887/

[142] E. T. Group, "EtherCAT–The Ethernet Fieldbus," 2020. [Online]. Available: https://www.ethercat.org/download/documents/ETG_Brochure_EN.pdf

[143] P. Consortium, "PROFINET–The solution platform for process automation,"2018. [Online]. Available: https://www.profibus.com/download/pi-white-paper-profinet-the-solution-platform-for-process-automation

[144] P. Pop, M. L. Raagaard, M. Gutierrez, and W. Steiner, "Enabling fog computing for industrial automation through time-sensitive networking," *IEEE Commun. Standards Mag.*, vol. 2, no. 2, pp. 55–61, Jun. 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8412464/

[145] V.-G. Nguyen, K.-J. Grinnemo, J. Taheri, and A. Brunstrom, "A deployable containerized 5G core solution for time critical communication in smart grid," in *Proc. IEEE 23rd Conf. Intell. Next Gener. Netw.*, 2020, pp. 153–155. [Online]. Available: https://ieeexplore.ieee.org/document/9059397/

[146] A. Larranaga, M. C. Lucas-Estan, I. Martinez, I. Val, and J. Gozalvez, "Analysis of 5G-TSN integration to support industry 4.0," in *Proc. 25th IEEE Int. Conf. Emerg. Technol. Factory Automat.*, 2020, pp. 1111–1114. [Online]. Available:https://ieeexplore.ieee.org/document/9212141/

[147] M. Gundall, C. Huber, P. Rost, R. Halfmann, and H. D. Schotten, "Integration of 5G with TSN as prerequisite for a highly flexible future industrial automation: Time synchronization based on IEEE 802.1AS," in *Proc. 46th Annu. Conf. IEEE Ind. Electron. Soc.*, 2020, pp. 3823–3830. [Online]. Available: https://ieeexplore.ieee.org/document/9254296/

[148] S. Qi, S. G. Kulkarni, and K. K. Ramakrishnan, "Assessing container network interface plugins: Functionality, performance, and scalability," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 656–671, Mar. 2021.

[149] S. S. N. Alhady and A. F. W. Othman, "Time-aware traffic shaper using time-based packet scheduling on intel I210," *Int. J. Res. Eng.*, vol. 5, no. 9, pp. 494–499, 2018. [Online]. Available: https://www.academia.edu/download/57723212/356_Time-aware_Traffic_Shaper_using_Time-based_Packet_Scheduling_on_Intel_I210_FINAL2.pdf

[150] D. B. D. Oliveira, D. Casini, R. S. D. Oliveira, and T. Cucinotta, "Demystifying the real-time linux scheduling latency," in *Proc. 32nd Euromicro Conf. RT Syst.*, 2020, pp. 9:1–9: 23. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2020/12372

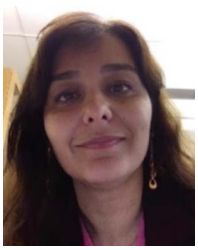
[151] C. S. V. Gutiérrez, L. U. S. Juan, I. Z. Ugarte, and V. M. Vilches, "Real-time linux communications: An evaluation of the Linux communication stack for real-time robotic applications," 2018, *arXiv: 1808.10821*.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.

[152] T. Kovacshazy, T. Tusori, and D. Vincze, "Prototype implementation and performance of time-based distributed scheduling on linux for real-time cyber-physical systems," in *Proc. IEEE Int. Symp. Precis. Clock Synchronization Meas. Control Commun.*, 2018, pp. 1–6. [Online]. Available:https://ieeexplore.ieee.org/document/8543070/
[153] C. Li, S. Xi, C. Lu, C. D. Gill, and R. Guerin, "Prioritizing soft real-time network traffic in virtualized hosts based on xen," in *Proc. 21st IEEE Real-Time Embedded Technol. Appl. Symp.*, 2015, pp. 145–156. [Online]. Available:http://ieeexplore.ieee.org/document/7108436/
[154] L. Lai, G. Ara, T. Cucinotta, K. Kondepu, and L. Valcarenghi, "Ultra-low latency NFV services using DPDK," in *Proc. IEEE Conf. Netw. Funct. Virtualization Softw. Defined Netw.*, 2021, pp. 8–14. [Online]. Available:https://ieeexplore.ieee.org/document/9665131/
[155] Y. Li, L. T. X. Phan, and B. T. Loo, "Network functions virtualization with soft real-time guarantees," in *Proc. 35th Annu. Int. Conf. Comput. Commun.*, 2016, pp. 1–9. [Online]. Available:https://ieeexplore.ieee.org/document/7524563/
[156] G. Ara, L. Lai, T. Cucinotta, L. Abeni, and C. Vitucci, "A framework for comparative evaluation of high-performance virtualized networking mechanisms," in *Proc. 10th Int. Conf. Cloud Comp. Serv. Sci.*, Springer International Publishing, 2021, pp. 59–83. [Online]. Available: https://link.springer.com/10.1007/978-3-030-72369-9_3
[157] C.-C. Tu, J. Stringer, and J. Pettit, "Building an extensible open vSwitch datapath," *ACM SIGOPS Operating Syst. Rev.*, vol. 51, no. 1, pp. 72–77, 2017. [Online]. Available:https://dl.acm.org/doi/10.1145/3139645.3139657
[158] W. Tu, J. Stringer, Y. Sun, and Y.-H. Wei, "Bringing the power of eBPF to open vSwitch," in *Proc. Linux Plumbers Conf.*, 2018, pp. 1–11. [Online]. Available: https://linuxplumbersconf.org/event/2/contributions/107/attachments/105/131/ovs-ebpf-afxdp.pdf
[159] F. Parola, R. Procopio, R. Querio, and F. Risso, "Comparing user space and in-kernel packet processing for edge data centers," *SIGCOMM Comput. Commun. Rev.*, vol. 53, no. 1, pp. 14–29, Apr. 2023, doi: 10.1145/3594255.3594257.



Remo Andreoli received the MSc degree with honors in computer science from the University of Pisa (Italy). He is currently working toward the PhD degree with Scuola Superiore Sant'Anna in Pisa (Italy). His research focuses on differentiated performance mechanisms for NoSQL databases and resource management optimization techniques for cloud infrastructures. He earned a best student paper award at CLOSER 2021.



Raquel Mini received the BSc, MSc, and PhD degree in computer science from the Federal University of Minas Gerais (UFMG), Brazil. She is a researcher with Ericsson, working in the Cloud Systems and Platforms area. Before joining Ericsson, Raquel worked for more than 20 years as a professor of computer Sscience in Brazil. Her research addresses the area of sensor networks, IoT, ubiquitous and cloud computing.



Per Skarin received the PhD degree from Lund University, Sweden. He recently joined SAAB as a senior research scientist. He has been a researcher and machine learning specialist with Ericsson for almost 11 years, working in the cloud technology and autonomous systems area.



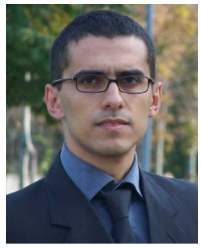
Harald Gustafsson received the MSc degree in electrical engineering from Lund University, and the PhD degree in applied signal processing from the Blekinge Institute of Technology, Sweden. He is an expert in network-compute application platforms with Ericsson Research, Lund, Sweden. His research addresses multimedia systems, distributed systems, communication systems and cloud systems on aspects as time criticality, reliability, and energy efficiency.



Janos Harmatos received the PhD degree in electrical engineering from the Budapest University of Technology and Economics, Hungary. He is currently a master researcher with Ericsson Research, investigating on deterministic communication and real-time cloud systems. He was involved in several standardization bodies, such as 3GPP, ETSI, and 5G-ACIA.



Luca Abeni revceived the MSc degree in computer engineering from the University of Pisa, and the PhD degree from Scuola Superiore Sant'Anna. He is a associate professor with Scuola Superiore Sant'Anna. He has also been researcher and associate professor with the University of Trento. His main research interests are operating systems, scheduling algorithms, QoS management, multimedia applications, and audio/video streaming.



Tommaso Cucinotta received the MSc degree in computer engineering from the University of Pisa, and the PhD degree in computer engineering from Scuola Superiore Sant'Anna. He is associate professor with Scuola Superiore Sant'Anna. His research interests include predictability in cloud computing and NFV. He has been MTS in Bell Labs in Dublin (Ireland), investigating on security and real-time performance of cloud services.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on October 04,2026 at 16:59:32 UTC from IEEE Xplore. Restrictions apply.